
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

E-COMMERCE APPLICATION DEVELOPMENT

Building, Running and Securing an Online Shop

UNIFIED LECTURE HANDOUT

Complete Course in One Volume • Fall 2026



Anyone can install a storefront in an afternoon. This course is everything after that.

A shop is not a website. It takes money, holds other people's data, and is answerable in law — and each of those changes what “finished” means.

What this volume covers. Every distinct clause of the HEC outline for *E-Commerce*, in the order the outline gives them, traced chapter by chapter in the Course Specification overleaf.

That outline is published once only, in the **2017** curriculum (p. 94), where the course is an *elective in BS Software Engineering* rather than a BSIT course at all. The 2023 curriculum drops it completely; the 2025 curriculum renames it *Digital Marketing & E-Commerce* and publishes no contents. The department teaches it as **INFT-4041**, BSIT, 5th semester, 3 credit hours. All of that is recorded in the Course Specification rather than smoothed over.

The course is taught on a **running shop**. One storefront is installed in Chapter 5 and every chapter afterwards adds a layer to it; by Chapter 23 it is the semester project. Appendix A sets up the stack in one sitting.

Six Parts • Twenty-Three Chapters • Seven Appendices

How to Use This Handout

There are two ways to teach this subject badly.

The first is to teach it as business studies: models, channels, trends, the history of the dot-com boom, and a graduate who can describe an online shop but has never configured a tax rule. The second is to teach it as programming exercises: write a cart class, write a checkout function, and a graduate who has built a toy that no one would take money through.

This handout takes the third path, which is also the one the published outline describes if you read it closely. You install a real, open-source storefront in Chapter 5, and you spend the rest of the course making it into a shop that could actually trade: a catalogue that models real products, a checkout that handles shipping and tax, a payment path that survives a failed transaction, an admin who can process a refund, a server that does not leak its database, and a listing a search engine can find. Where the platform cannot do what you need — and in Chapter 19 it cannot — you write the code that extends it.

The six parts

Part I — E-Commerce and Its Models (Chapters 1–4) is the business grounding, and it is short on purpose. What electronic commerce actually changed; the models and where the revenue comes from; auctions, portals and communities; and the build-buy-or-extend decision that justifies everything the rest of the book does.

Part II — Standing the Shop Up (Chapters 5–8) gets a storefront running and fills it. The stack and what each layer does; products and categories, with the tables underneath them; variants, options and uploads; then the theme.

Part III — From Basket to Order (Chapters 9–12) is the pipeline a customer walks through. Where basket state lives; the checkout and the order state machine; shipping, tax and zones; discounts, vouchers and referrals.

Part IV — Money and Identity (Chapters 13–16) is the part where mistakes cost money. Payment systems and who carries the risk; integrating a gateway and testing it in a sandbox; accounts, sessions and password storage; then securing the whole thing.

Part V — Running and Extending the Shop (Chapters 17–20) is the half of the job nobody demonstrates: the dashboard and what to watch, the back office, writing your first module, and deployment with everything that follows it.

Part VI — Reach, Law and Practice (Chapters 21–23) closes with being found, being lawful, and your project.



Why Chapter 19 is the hinge of the course. Everything before it is *configuration*: you are choosing among the options a platform already offers, and a careful non-programmer could do most of it. Chapter 19 is the first time the answer to “can the shop do this?” is no, and you have to write the code that makes it yes. That is the difference between the outline’s bare title, *E-Commerce*, and the one this department teaches, *E-Commerce Application Development*.

The furniture of every chapter

Box	What it is for
🎯 Outcomes	What you will be able to <i>do</i> . Bloom verbs, not topics.
🔗 Before you start	Which earlier chapters this one stands on, and what state your shop should be in.
🔑 Key terms	The vocabulary introduced; all of it is in the glossary of Appendix D.
📖 Definition	The canonical statement. In this subject several ordinary English words — order, basket, customer, product — are technical terms with narrower meanings, and confusing the two causes real bugs.
💡 Concept	The judgement call: what an experienced practitioner would decide, and why.
⚠️ Warning	Something that will bite you, usually in production rather than in the examination.
📊 Worked example	Real arithmetic — a tax calculation, a gateway fee, a discount applied in the wrong order — with every step shown.
📏 Measured	A number produced by running something, not by asserting it. Twelve chapters carry one; see below.
❗ Common pitfalls	The mistakes that are actually made.
🧪 Try it yourself	The laboratory. Every one of them changes the same shop, and they are cumulative: skip Chapter 9's and Chapter 10's will have nothing to check out.
❓ Checkpoint	Three questions. Attempt them before reading Appendix C.
✅ Chapter summary	What to retain.
🔧 Review questions	Examination practice: multiple choice, short answer, applied.

Two typographic conventions carry a lot of weight in a course taught through a graphical interface, and they are worth learning on this page:

- **Catalog > Products > Add New** is a **click path** through the administration panel. Read the chevrons as “then”.
- **Save** on its own is a single control — a button, a field, a tab or a menu item. Anything in **typewriter** is text you type or a file you edit, which is a different thing entirely.

The measured strand, and why it is only in twelve chapters

Twelve of the twenty-three chapters end with a number that was produced rather than asserted: a page time, a query count, a request rate, a hash cost, a list of security headers present and absent.

They are the twelve where measuring genuinely teaches something. “Add an index and the category page gets faster” is a claim a student can be told and will forget; watching the same page go from a table scan at fifty thousand products to an index seek, and reading both numbers off your own machine, is not. The other eleven chapters argue in prose, because dressing up a legal obligation or a business model with a stopwatch would be theatre.

Every figure in a  **Measured** box was produced by the file named beside it, in `code/`, against the stack in Appendix A. Rerun them. Your numbers will differ from the book’s — different machine, different disk, different day — and the *shape* should not. Where a result depends on the particular machine, the box says so.

Course Specification

This page is the course file. It states what HEC requires of a course called *E-Commerce* and shows where each requirement is met.

That is harder here than in most courses, because the requirement has been published exactly once and then withdrawn. The three editions are traced below so that an accreditation review against any of them has something to check.

Course particulars

Item	Value
Course title	<i>E-Commerce Application Development</i> , as taught. HEC 2017 calls it simply <i>E-Commerce</i> ; HEC 2025 calls it <i>Digital Marketing & E-Commerce</i> . The department's title is the accurate one, and the Course Specification justifies the two added words
Course code	INFT-4041
Programme	BS Information Technology, 5th semester. Under HEC 2017 the course is an elective in BS Software Engineering (2017, p. 38, “select any FIVE”) and does not appear in the BSIT scheme at all
Credit hours	3: three credit hours of lectures, no separate laboratory credit. The laboratory work in this handout is therefore done in lecture time and as homework, which is why every lab is small and cumulative
Prerequisite	<i>Web Technologies</i> (CS-207 / INFT-4005, 4th semester). HEC 2017 names the prerequisite <i>Web Engineering</i> ; the department's course of that content is called <i>Web Technologies</i> . The name differs, the dependency is satisfied — see Appendix F
Duration	Sixteen teaching weeks, midterm in Week 8, project due Week 16 (Appendix G)
Teaching methodology	As published (HEC 2017): lecturing, written assignments, project , report writing
Assessment	As published: sessional examination, home assignments, quizzes, project, presentations, final examination
Set texts	As published: Laudon and Traver, <i>E-Commerce</i> ; Peacock, <i>PHP 5 E-commerce Development</i> ; Rayport, <i>Introduction to E-Commerce</i> ; Schneider, <i>Electronic Commerce</i> . See defect 5 below, and Appendix E

The three editions

Ed.	Title	Status	Contents
2017	E-Commerce	Elective in BS Software Engineering (p. 38); absent from BSIT. 3(3,0); prerequisite <i>Web Engineering</i>	Yes, p. 94 — the only published outline in any edition
2023	—	Absent entirely. The course appears nowhere in the computing-disciplines curriculum except as an incidental application domain (p. 23)	No
2025	Digital Marketing & E-Commerce	Interdisciplinary / Allied course, item 11 (p. 16). Not a computing core; universities may add others	No — title and credit hours only

Five defects in the published document

Recorded here so that a reader comparing this handout with the HEC booklet is not confused, and so that the department's own course file does not inherit them.

1. Two learning outcomes, both too broad to assess. HEC publishes exactly two: “Understand the concepts and standards related to the discipline of E-Commerce” and “Analyze complex real world problems found in E-Commerce”. Sibling courses publish eight. Two cannot carry sixteen weeks or a question paper, so this department adds six, each traced back to one of HEC's two and each marked as departmental in the table below.

2. The Bloom's Taxonomy column is blank for both outcomes — only the Domain column (“C”) is filled. The levels below are therefore *assigned by this department*, and are marked as such.

3. The “Course Content” paragraph is the table of contents of its own reference book. Clause for clause, in order, it is the chapter list of Michael Peacock's *PHP 5 E-commerce Development* (Packt, 2010), which is reference 2 on the same page. HEC did not write an outline for this course; it pasted a contents page.

4. Two clauses are duplicated, which is what pasting a contents page does. “Checkout” appears twice — once as “The Checkout and Order Process” — and “Managing Products and Categories” appears twice, once on its own and once under Administration. Twenty-four clauses; twenty-two distinct.

5. The prescribed technology is end-of-life and the editions are stale. PHP 5 lost security support on 31 December 2018: the prescribed text targets a runtime that has been unsafe to deploy for longer than most of this class has been studying. Laudon and Traver is cited at the 13th edition (2017) and Rayport at 2007. Appendix E cites current editions and explains each gap.

Coverage of the HEC 2017 course outline

The published outline is a single run-on paragraph. Each of its clauses is traced below, in the order HEC writes them. Nothing is omitted, and the two duplicates are shown where they fall rather than quietly merged.

HEC 2017 clause	Chapter	Where exactly
“An overview of E-Commerce & its business models and concepts”	1, 2	What changed and why; then the models, with the revenue model treated as the question that decides the rest
“Planning an E-Commerce Framework”	4	Requirements, and the build-buy-or-extend decision this course’s answer rests on
“Managing Products and Categories”	6	The admin screens, and the tables beneath them
“Product Variations and User Uploads”	7	Options and variants, the combinatorics they generate, and why an upload field is a security question
“Enhancing the User Experience”	8	Themes and template overrides, measured by page weight rather than taste
“The Shopping Basket”	9	Session, cookie or database, and what survives a restart
“The Checkout and Order Process”	10	The order state machine, step by step
“Shipping and Tax”	11	Geo zones, tax classes, and sales tax on a Pakistani online sale
“Discounts, Vouchers, and Referrals”	12	All three, and the order in which they must be applied
“Checkout” (<i>duplicate of clause 7</i>)	10	Same chapter; see defect 4
“Taking Payment for Orders”	14	The gateway handshake, the sandbox, and what happens when it fails halfway
“User Account Management”	15	Registration, sessions, password storage, reset flows
“Administration: Dashboard”	17	The dashboard, and which of its numbers are worth acting on
“Managing Products and Categories” (<i>duplicate of clause 3</i>)	6, 17	Same material from the back office; see defect 4
“Managing Orders”	18	Order states, fulfilment, partial shipment
“Customers”	18	Customer records, groups and approval
“Refunds”	18	Returns, credit and what the order state machine does next
“Voucher Codes”	12, 18	Created in 12, administered in 18
“Shipping” (<i>administration</i>)	11, 18	Configured in 11, operated in 18
“Deploying, Security, and Maintenance”	16, 20	Security is given its own chapter — see the note below — and deployment, backup and maintenance share 20
“Web Payment Systems”	13	Cards, wallets, bank transfer and cash on delivery; cost and risk for each

HEC 2017 clause	Chapter	Where exactly
“Social, Legal, and Ethical Issues of E-Commerce”	22	Consumer rights, privacy, dark patterns, accessibility, and the Pakistani statute
“Auctions, Portals, and Communities”	3	Moved forward to Part I, where it belongs: these are business models
“SEO”	21	Crawlability, URL structure, metadata, sitemaps

Course learning outcomes

The two outcomes HEC publishes, verbatim, followed by the six this department adds. Bloom levels are assigned by this department throughout, because the published table leaves the column blank.

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-1	<i>(HEC, verbatim)</i> Understand the concepts and standards related to the discipline of E-Commerce	C2 Understand	1–4, 13, 22
CLO-2	<i>(HEC, verbatim)</i> Analyze complex real world problems found in E-Commerce	C4 Analyse	6, 10, 16, 20, 23
CLO-3	<i>(dept.)</i> Explain e-commerce business and revenue models, and classify a given venture against them	C2 Understand	2, 3
CLO-4	<i>(dept.)</i> Install, configure and secure an open-source storefront on a reproducible stack	C3 Apply	5, 16, 20
CLO-5	<i>(dept.)</i> Model a catalogue — products, categories, options and variants — and justify the schema that results	C3 Apply	6, 7
CLO-6	<i>(dept.)</i> Configure the order pipeline end to end: basket, checkout, shipping, tax, discounts and payment	C3 Apply	9–12, 14
CLO-7	<i>(dept.)</i> Analyse a storefront’s performance and security posture from measurement rather than assertion	C4 Analyse	6, 8, 15, 16, 20, 21
CLO-8	<i>(dept.)</i> Extend the platform with a module meeting a stated requirement, then deploy, present and defend the result	C5 Evaluate	19, 20, 23

💡 Three topics taught that the outline does not name

Each is justified here rather than smuggled in, because a reviewer will ask.

Chapter 6’s data model. The outline names the screens — “Managing Products and Categories” — but never the tables. A graduate who can fill in an admin form but cannot say why a product’s name lives in a different table from its price has not understood the catalogue, and *Database Systems* is a prerequisite that ought to be paid off rather than left on the shelf.

Chapter 16, Security, as a chapter of its own. HEC buries it inside “Deploying, Security, and Maintenance”, one clause among three. A shop holds addresses, order histories and password hashes, and CLO-7 asks for a security posture to be *analysed*. One third of one clause will not do it.

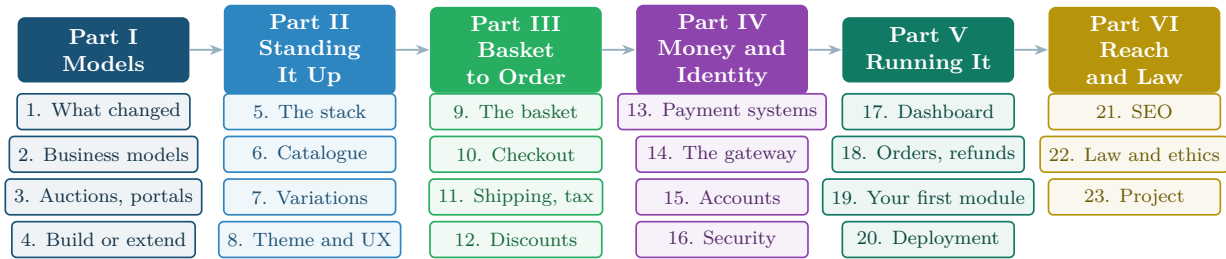
Chapter 19, module development. The outline is configuration from end to end; nothing in it requires a student to write a line of code. The department’s title says *Application Development*, the HEC teaching methodology requires a project, and a project that is entirely configuration is not a computing project.

⚠ Where this course sits, and the one awkwardness

The outline this handout traces was written for *software engineering* students as an elective, and it shows: it assumes the reader will build a cart from scratch in PHP. This department teaches it to *information technology* students as a 5th-semester course, where the useful skill is standing up, configuring, securing and extending a real platform rather than reimplementing one badly.

This handout therefore follows the outline’s *clauses* faithfully and its *method* not at all. Every topic HEC lists is taught; none of it is taught by writing a shopping cart in PHP 5. Chapter 4 makes that argument explicitly, so that the student can defend the choice rather than merely inherit it.

The Course at a Glance



 One shop, installed in Chapter 5 and added to every week until it is the project in Chapter 23.

Figure F.1: The six parts and the chapters in each. Chapter 19 is the hinge: it is the first chapter in which configuring the platform is not enough and you have to write code.

This course stands on Web Technologies and Database Systems, and shares its deployment ground with Virtual Systems and Services in the same semester. Appendix F states what belongs to each.

The Shop We Build

Every lab in this book changes the same shop. Nothing is thrown away, and nothing is built twice. This page is the map of what gets added when, so that you can tell at any point in the semester whether you are behind.

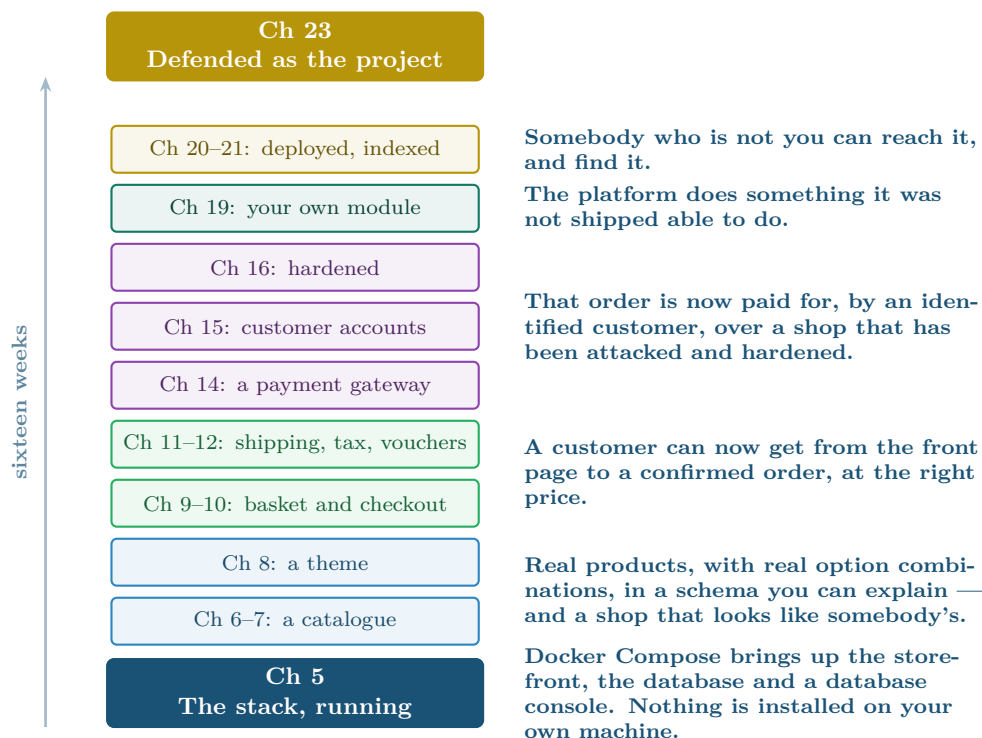


Figure F.2: What each part adds to the one shop. A lab skipped is a lab the next chapter cannot build on — Chapter 10 has nothing to check out if Chapter 9's basket was never configured.

⚠ Before Week 2

The stack runs in containers, so nothing in this course asks you to install a web server, PHP or a database on your own machine. It does ask you to install **one** thing: a container runtime. Appendix A does it in one sitting and lists what to do when it will not start, which on a university laptop it sometimes will not.

Do it before Week 2. From Chapter 5 onward every chapter assumes a shop you can reach in a browser, and a student without one spends the semester reading about a course the rest of the class is taking.

Prerequisite Self-Check

Twelve questions. If you can answer ten, begin at Chapter 1. If you cannot, Appendix B covers the PHP, SQL and template syntax this book assumes, your *Web Technologies* and *Database Systems* notes cover the rest — and the gap is worth closing in Week 1 rather than Week 9.

1. What is the difference between an HTTP GET and an HTTP POST, and which one must a form that charges a card use? Why? (Web Technologies)
2. What do the status codes 200, 302, 404 and 500 mean, and which one does a browser follow without telling the user? (Web Technologies)
3. HTTP is stateless. How, then, does a server know that two requests came from the same visitor? (Web Technologies, Chapter 9)
4. Write a **SELECT** that joins two tables and returns one row per match. (Database Systems, Appendix B)
5. What is a database index for, and what does adding one cost? (Database Systems, Chapter 6)
6. Distinguish a primary key from a foreign key, and say what the database guarantees about each. (Database Systems)
7. One product has many photographs. Which table holds the photographs, and why not the product table? (Database Systems, Chapter 6)
8. In an HTML form, what does the **name** attribute do, and where does its value arrive on the server? (Web Technologies)
9. A theme sets a heading to 18px and you want 24px. Give two ways to override it in CSS, and say which one you would not use. (Web Technologies, Chapter 8)
10. In PHP, what does `$_POST['qty']` contain, and why may you never use it in a query without doing something to it first? (Appendix B, Chapter ??)
11. What does HTTPS protect, and name one thing it does *not* protect. (Information Security, Chapter ??)
12. Open a terminal in a named directory, run a command, and read its output. If that sentence is not routine, start with Appendix A. (Appendix A)



Notice what is *not* on that list. You do not need to be a strong PHP programmer: until Chapter 19 you will read far more code than you write, and Appendix B gives you enough syntax to read it. You do not need any business background. You do not need a powerful machine.

What you do need is the habit of *checking* — opening the database to see what the form actually wrote, reading the error log rather than guessing, rerunning a measurement after changing one thing. Most of what goes wrong in this course is invisible from the front page of the shop, and every chapter tries to teach you where to look.

Contents

How to Use This Handout	ii
Course Specification	v
The Course at a Glance	x
The Shop We Build	xi
Prerequisite Self-Check	xii
I E-Commerce and Its Models	1
1 What E-Commerce Is, and What It Changed	3
1.1 A Shop With No Closing Time	3
1.2 The Four Functions	4
1.3 What Actually Changed	4
1.4 Why This Is a Software Problem	5
1.5 Reading a Shop You Have Not Seen Before	6
2 Business Models and Revenue Models	9
2.1 Two Questions That Get Confused	9
2.2 Classifying by Who Is at Each End	10
2.3 The Revenue Models	10
2.4 The Arithmetic of One Order	11
3 Auctions, Portals and Communities	15
3.1 Why This Chapter Is Here Rather Than at the End	15
3.2 Letting the Buyer Set the Price	15
3.3 What an Auction Costs You in Engineering	16
3.4 Holding Other People's Money	17
3.5 Portals, Marketplaces and Communities	17
4 Planning an E-Commerce Framework	21
4.1 The Question This Chapter Answers	21
4.2 The Four Options, Honestly	22
4.3 Requirements That Actually Discriminate	23
4.4 The Cost That Is Not on the Price List	23
4.5 Making the Decision Defensible	24

II	Standing the Shop Up	28
5	The Stack, and Your Shop in One Command	30
5.1	Four Layers, and What Each One Does	30
5.2	Containers, and What They Are Not	31
5.3	The Three Services in This Course's Stack	32
5.4	What Happens the First Time	33
5.5	Looking At What You Were Given	34
6	Products, Categories and Their Tables	37
6.1	One Product Is Not One Row	37
6.2	The Screen, and What It Writes	38
6.3	Categories, and the Table That Is Not Normalised	39
6.4	What the Category Page Actually Asks	40
6.5	What It Costs, and What an Index Buys	41
7	Product Variations, Options and Uploads	46
7.1	Two Ways to Say "Comes in Three Colours"	46
7.2	What Each Choice Costs	48
7.3	Uploads, and What Your Shop Will Accept	48
8	Themes, Layout and the Customer's Experience	53
8.1	What a Theme Actually Is	53
8.2	Layouts, or How to Change a Page Without Editing It	54
8.3	What a Page Weighs	54
8.4	Judging a Change by the Customer	56
III	From Basket to Order	59
9	The Shopping Basket, and Where State Lives	61
9.1	The Problem: A Protocol With No Memory	61
9.2	Three Places State Can Live	62
9.3	Where This Shop Actually Puts It	62
9.4	What a Session Costs	63
9.5	Two Things the Source Does That the Screens Do Not Show	64
9.6	What Survives	65
9.7	The Settings That Govern Any of This	66
	Appendices	69
A	The Stack: Setting It Up and Keeping It	70
A.1	What You Install	70
A.2	Starting the Shop	70
A.3	The Four Commands Worth Memorising	71

A.4	When It Will Not Start	71
A.5	Backing Up, and Why You Will Wish You Had	71
A.6	The Machine Every Measurement Was Taken On	72
B	Enough PHP, SQL and Twig to Read the Shop	73
B.1	PHP, in Two Pages	73
B.2	SQL, as This Course Uses It	75
B.3	Twig, Which Is Just the Page	76
B.4	YAML, in Ten Lines	76
B.5	Where To Look Next	77
C	Answers to the Checkpoints	78
D	The Sixteen-Week Schedule	83
D.1	The Schedule	83
D.2	How the Assessment Adds Up	84

List of Figures

F.1	The six parts and the chapters in each. Chapter 19 is the hinge: it is the first chapter in which configuring the platform is not enough and you have to write code.	x
F.2	What each part adds to the one shop. A lab skipped is a lab the next chapter cannot build on — Chapter 10 has nothing to check out if Chapter 9’s basket was never configured.	xi
P1.1	Reading path through Part I. Chapters 1–3 are the business grounding; Chapter 4 turns it into an engineering decision, and is the one to argue with.	2
1.1	The four functions of any shop, and the parts of this book that build them. The dashed line is the one that matters: everything to its left can be undone by closing the browser tab, and nothing to its right can.	4
2.1	The five revenue models a shop can use, and the single fact about each that most shapes the software. Most real businesses run two of them at once.	10
4.1	The four ways to obtain a shop. They are a spectrum, not a menu: each step to the right buys control and costs effort, and the right answer depends on which of the two you are shorter of.	21
P2.1	Reading path through Part II. Chapter 6 is the one to slow down on: it is the only chapter that looks at the same thing from the admin panel and from the database at once.	29
5.1	The four layers. The traditional name for this arrangement is LAMP — Linux, Apache, MySQL, PHP — and although every letter has alternatives now, the shape has not changed in twenty-five years.	31
P3.1	Reading path through Part III. Chapter 10 is the hinge of the whole book — an order is the first object in the course that cannot simply be deleted and redone. . .	60
9.1	The three stores, and what each one buys. Nothing forces a shop to pick one: most use all three at once, for different things. The question worth asking of any shop is not which one it uses, but which one holds the basket.	62

PART I

E-Commerce and Its Models

Before you install anything, four questions have to have answers: what electronic commerce actually changed, how a shop makes money, what shapes other than a shop exist, and — given all that — whether you should build a storefront, buy one, or extend one.

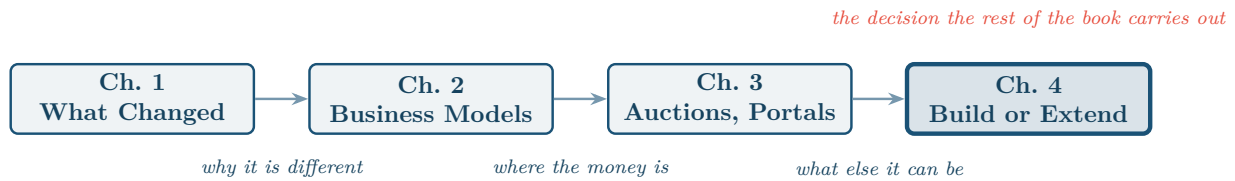


Figure P1.1: Reading path through Part I. Chapters 1–3 are the business grounding; Chapter 4 turns it into an engineering decision, and is the one to argue with.

What E-Commerce Is, and What It Changed

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — E-Commerce and Its Models

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** electronic commerce, and **distinguish** it from the wider category of electronic business.
- **Name** the four functions every shop must perform, and **locate** each of them in a storefront you have not seen before.
- **Explain** why selling online is a software problem rather than a marketing one, giving two consequences that follow.
- **Distinguish** the parts of a transaction that electronic commerce actually changed from the parts it only moved.
- **Classify** a given online business by the four functions, and **identify** which of them it has chosen not to perform itself.

🔗 Before You Start

Nothing in this book. You need to have used an online shop as a customer, which is the only prerequisite this chapter has. Appendix A sets up the software you will need from Chapter 5 onward; start it this week, because it sometimes needs a second attempt.

📖 Key Terms in This Chapter

electronic commerce • electronic business • storefront • catalogue • basket • checkout • fulfilment • transaction • disintermediation • long tail • friction • conversion • abandonment • trust

1.1 A Shop With No Closing Time

Consider a bookshop on a street in Bahawalpur. It opens at nine and closes at nine. It holds perhaps four thousand titles, because that is what the shelves take. Its customers are the people who can reach the street. If you want a book it does not stock, the shopkeeper can order it, and you will come back next week.

Now consider the same bookshop online. It is open at three in the morning. It can list four hundred thousand titles, because a listing is a database row and rows are cheap. Its customers are anyone with a connection. If you want a book it does not stock, the website can know that instantly, and can offer to tell you when it arrives.

Every one of those differences is real, and none of them is the interesting one.

The interesting difference is this: in the street shop, a human being stands between the customer and every mistake. If the price label is wrong, the shopkeeper notices. If you try to buy two of something when one is left, the shopkeeper tells you. If you hand over a torn note, the shopkeeper decides. The shop runs on a continuous supply of human judgement, and that judgement is invisible precisely because it never fails conspicuously.

Online, there is no shopkeeper. Every one of those judgements has to have been written down, in advance, as a rule — and anything that was not written down will happen.

Electronic Commerce

Electronic commerce is the buying and selling of goods and services over a computer network, together with the transfer of money and data that completes those transactions.

It is a subset of **electronic business**, which is the use of networks for *any* business process — a supplier portal, an internal stock system, a staff payroll. The distinguishing feature of e-commerce is the **transaction**: value changes hands, and both parties are entitled to expect that it did so exactly once.

That last clause is not pedantry. It is the reason Chapter ?? is the longest chapter in this book.

1.2 The Four Functions

Strip away the branding and every shop on the internet does the same four things, in the same order. This is worth fixing now, because it is the skeleton of the whole course: each function is a part of this book, and each is a cluster of screens in the software you will install in Chapter 5.

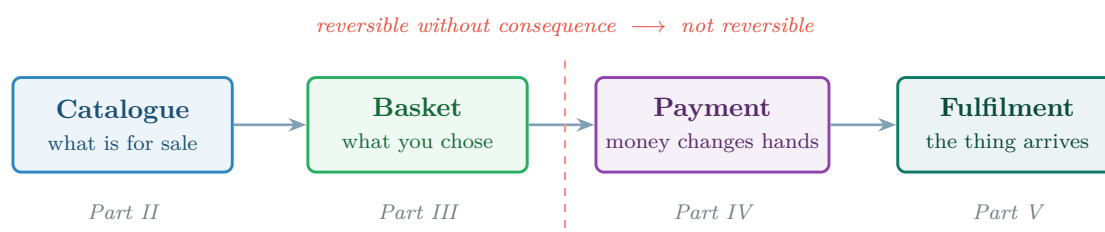


Figure 1.1: The four functions of any shop, and the parts of this book that build them. The dashed line is the one that matters: everything to its left can be undone by closing the browser tab, and nothing to its right can.

The dashed line is the whole subject

A customer browsing a catalogue and filling a basket has committed to nothing. They can close the tab and the shop is exactly as it was. Mistakes on that side of the line cost a sale at worst.

The moment payment is attempted, that stops being true. Money may have moved while the network dropped. An order may exist that the customer does not know about, or the customer may believe in an order that does not exist. Stock may have been reserved for someone who never paid.

Most of what distinguishes a real shop from a student exercise is the handling of that line, and most student projects put all their effort on the left of it because that is the half you can see.

1.3 What Actually Changed

It is easy to overstate what e-commerce changed. A great deal of it is the same commerce, conducted through a different window. Three things, though, genuinely changed, and they explain most of what the rest of this book worries about.

1.3.1 The cost of listing fell to nearly nothing

Shelf space is expensive and finite; a database row is neither. A street shop must stock what sells often, because an item that sells twice a year occupies the same shelf as one that sells twice a week. An online shop has no such pressure, and can profitably carry items that sell twice a year — the **long tail**.

That is a business observation with an engineering consequence that this course will measure: a catalogue of four hundred thousand items does not behave like a catalogue of four thousand. Chapter 6 puts fifty thousand products into your shop and times the result, and the answer is not comfortable.

1.3.2 Some middlemen became unnecessary, and others appeared

Disintermediation is the removal of intermediaries: a manufacturer can sell directly to you, and the distributor and the retailer are not needed.

The prediction made in the 1990s was that this would flatten commerce. What happened instead was that old intermediaries were replaced by new and larger ones — marketplaces, payment processors, delivery networks, search engines. A shop today typically pays a marketplace for access to customers, a gateway for the ability to take money, and a courier to deliver, and each of those is a dependency with a price and a failure mode. Chapter ?? is about one of them in detail.

1.3.3 Trust had to be manufactured

In a street shop, trust is cheap. The shop is physically there; you can see the goods; if it cheats you, you know where it is.

Online, none of that holds, and every substitute has to be built deliberately: a visible address and a returns policy, a padlock in the address bar, reviews, an order confirmation that arrives within seconds, a refund that works. These are not decoration — they are the mechanism that replaces a shopfront, and each of them is a feature somebody has to implement and maintain.

✎ Three signals, and what each costs to fake

A customer deciding whether to buy from an unfamiliar shop is reading signals. It is worth knowing which ones are hard to fake and which are not.

- **A professional-looking design.** Costs almost nothing to fake; a template is free. Carries little information.
- **A valid certificate and HTTPS.** Costs nothing now that certificates are free — which is why its absence is damning and its presence proves very little. Chapter ?? returns to exactly what it does and does not prove.
- **A confirmation email, with an order number, that arrives in four seconds.** Hard to fake, because it requires the shop to actually have the machinery. This is one reason Chapter ?? treats the confirmation as part of the transaction rather than as an afterthought.

1.4 Why This Is a Software Problem

The common way to teach this subject is as business studies with screenshots. That gets the emphasis wrong, and the published outline this course follows — see the Course Specification — gets it right: eighteen of its twenty-two clauses are about building, configuring and running the software.

Here is the argument in one paragraph. A shop is a set of rules that must hold without supervision: this price applies to this customer in this quantity in this region on this date; this voucher may be used once; this stock is reserved for eleven minutes; this payment either completed or it did not. Every one of those is a statement about state, every one of them has an edge, and nobody is watching. That is not a marketing problem. It is the same problem as any other concurrent, stateful, partially-trusted system, and it is solved with the same tools.

⚠ The two failures that are specific to this subject

Almost everything that goes badly wrong in an online shop is one of two things, and both are invisible from the front page.

The state got out of step. Money taken, no order recorded. Order recorded, stock never reduced. Refund issued twice. These are not rare, and they do not announce themselves — the shop looks fine, and the accounts do not balance at the end of the month. Chapter ?? and Chapter ?? are about not causing them.

Something was trusted that should not have been. A price taken from a form field rather than from the database. A quantity that was allowed to be negative. An administrative page that was never actually checked for a login. Chapter ?? is a whole chapter about this, and its laboratory finds several of them in the shop you will have built by then.

1.5 Reading a Shop You Have Not Seen Before

The skill this chapter is really teaching is a diagnostic one, and it is worth practising before you build anything. Given any online business, you should be able to say within a few minutes which of the four functions it performs itself, which it has handed to somebody else, and what that choice costs it.

📊 Worked Example: Classifying three businesses by the four functions

A clothing brand selling from its own site. Catalogue: its own. Basket: its own. Payment: a gateway — it does not hold a banking licence, so this is handed over. Fulfilment: usually a courier. *Performs two of four; the two that define it.*

The same brand selling only through a large marketplace. Catalogue: the marketplace's, in the marketplace's format. Basket: the marketplace's. Payment: the marketplace's. Fulfilment: possibly the marketplace's warehouse. *Performs none of the four.* It has bought access to customers and sold its relationship with them; it does not know who they are, cannot email them, and can be delisted. The commission is the visible price and is not the largest one.

A food delivery platform. Catalogue: aggregated from restaurants. Basket: its own. Payment: its own. Fulfilment: its own riders — which is the expensive one, and the one that is hard to copy. *Performs three and a half of four*, and the half it does not perform (the cooking) is the one with the thinnest margin.

The general rule. The function a business insists on performing itself is usually the one it believes is its advantage. Ask which function a business refuses to outsource, and you have found what it thinks it is.

⚠ Common Pitfalls

- **Confusing e-commerce with having a website.** A site that shows a catalogue and a phone number is a brochure. The transaction is the thing; without it, none of the hard problems in this book arise.
- **Assuming the hard part is the shopping cart.** It is not, and it has not been for twenty years — you will configure one in an afternoon in Chapter 9. The hard parts are everything after the dashed line in Figure 1.1.
- **Treating trust signals as design.** A returns policy is not a page; it is a promise that somebody has to keep, and a refund path somebody has to build (Chapter ??).
- **Believing disintermediation removed the middlemen.** It replaced them. Count your dependencies before claiming you have none.

- **Saying “we will add payments later”.** Everything to the right of the dashed line constrains everything to its left. Deciding how you take money late is how projects discover their data model is wrong.

Try It Yourself: Reading four real shops

No software is needed for this one. Chapter 5 installs the stack; this week you are learning to look.

Choose **four** online businesses that you can actually reach, of deliberately different shapes — for example a single-brand store, a large general marketplace, a services business that takes bookings rather than shipping goods, and a local business selling through social media only.

For each one, as a customer and without buying anything, record:

1. **The four functions.** Who performs each — the business, or somebody else? How can you tell? For payment, the giveaway is usually whether the address bar changes during checkout.
2. **Where the catalogue ends.** How many categories deep does it go? Can a product appear in two categories? Does it have options — size, colour — and are they separate products or one product with choices? You are looking at a data model through a window; Chapter 6 will show you the model itself.
3. **The furthest point you can reach without an account.** Can you fill a basket? Reach the checkout? See the shipping cost before committing? Write down the exact step at which it stopped.
4. **Three trust signals,** and for each, whether it is cheap or expensive to fake.
5. **One thing that is broken or confusing.** There is always one. Describe it precisely enough that somebody could fix it.

Write up one page comparing the four, ending with a single sentence for each: what does this business think it is? Keep it — Chapter 4 asks you to do the same exercise for the shop you are going to build, and it is much easier the second time.

What to notice. The four shops will differ far more in what they *refuse* to do than in what they do. Everyone has a catalogue. Not everyone will tell you the delivery charge before you have handed over your address, and the ones that will not have usually decided that on purpose.

Checkpoint — Test Yourself

1. A university lets students view results and download transcripts online, but fee payment happens at a bank counter. Is this electronic commerce, electronic business, both or neither? Justify your answer from the definitions.
2. Give one judgement a human shopkeeper makes without noticing, and describe precisely what has to exist in software to replace it.
3. A marketplace charges a 15% commission. A seller calculates that running its own site would cost less than 15% of revenue and concludes it should leave. Name two costs the seller has not counted, and one thing it would gain that is not money.

Chapter Summary

- Electronic commerce is the buying and selling of goods and services over a network, *including* the transfer of money. The transaction is what distinguishes it from electronic business generally.

- Every shop performs four functions in order: catalogue, basket, payment, fulfilment. They are Parts II to V of this book.
- The line between basket and payment is the one that matters. Everything before it is reversible by closing a tab; nothing after it is.
- Three things genuinely changed: the cost of listing collapsed, old intermediaries were replaced by new and larger ones, and trust stopped being free and had to be manufactured.
- The absent shopkeeper is the engineering problem. Every judgement a human would have made has to have been written down in advance, and whatever was not written down will eventually happen.
- Almost all serious failures are one of two kinds: state that got out of step, or something trusted that should not have been.
- To read an unfamiliar business, ask which of the four functions it performs itself. The one it refuses to outsource is what it believes it is.

Review Questions

1. **(MCQ)** Which of these is electronic business but *not* electronic commerce? (a) A customer buying a phone from a web shop; (b) a supplier checking a manufacturer's stock levels through a portal; (c) a customer paying a utility bill through an app; (d) a marketplace taking commission on a sale.
2. **(MCQ)** In Figure 1.1, the dashed line separates: (a) front end from back end; (b) free actions from paid actions; (c) what a customer can undo by closing the tab from what they cannot; (d) catalogue data from order data.
3. **(Short)** Explain the long tail, and give one engineering consequence of it that a shop owner would not anticipate.
4. **(Short)** "Disintermediation removed the middleman." Argue against this with a concrete example.
5. **(Short)** A friend says the hard part of building an online shop is the shopping cart. Give the strongest version of why they are wrong, and name the two failure modes you would be more worried about.
6. **(Applied)** A bookshop with four thousand titles in a shop wants to list eighty thousand online, most of which it does not hold in stock. Identify three decisions this forces that would not arise at four thousand titles, and say for each which of the four functions it affects.
7. **(Applied)** Take any one of the four businesses from this chapter's laboratory. Describe, in enough detail that it could be checked, an experiment a customer could run from outside that would reveal whether the shop reserves stock during checkout. State what result would mean "yes", what would mean "no", and what would be inconclusive.

Business Models and Revenue Models

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — E-Commerce and Its Models

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** a business model from a revenue model, and **explain** why a venture can have a sound one and a hopeless other.
- **Classify** an online business by participant — B2C, B2B, C2C, C2B — and say what each classification predicts about the software.
- **Name** the principal revenue models and **identify** which one a given shop is using.
- **Explain** the unit economics of a single order, and **compute** the contribution margin from a set of figures.
- **Argue** which revenue model a given venture should adopt, and **state** the software consequences of that choice.

🔗 Before You Start

Chapter 1: the four functions, and the habit of asking which of them a business performs itself.

📖 Key Terms in This Chapter

business model • revenue model • value proposition • B2C • B2B • C2C • C2B • marketplace • commission • subscription • freemium • advertising • affiliate • unit economics • contribution margin • customer acquisition cost • lifetime value • network effect

2.1 Two Questions That Get Confused

Almost every failed online venture answered one of these two questions well and the other not at all.

📑 Business Model and Revenue Model

A **business model** says *what value you create and for whom*: who the customer is, what problem is solved, and why they would choose you rather than the alternative. It is the answer to “why does this deserve to exist?”

A **revenue model** says *how the money reaches you*: who pays, for what, how often, and how much. It is the answer to “who writes the cheque?”

They are independent. The same business model — say, connecting people who want a ride with people who will drive — can be monetised by commission per ride, by a driver subscription, by advertising, or by charging nothing and selling the data. Those four are different companies with different software.

The confusion matters because the two fail differently and the failures look alike from outside. A venture with no business model and a clear revenue model builds something nobody wants and charges correctly for it. A venture with a strong business model and no revenue model builds something people love and cannot pay for it — and it is usually the second that students build.

2.2 Classifying by Who Is at Each End

The oldest classification asks only who is transacting with whom. It is coarse, and it is still the first question worth asking, because it predicts a surprising amount about the software.

Type	Who to whom	What it implies for the software
B2C	A business sells to a consumer	Many small orders, card and wallet payment, a catalogue tuned for browsing, and heavy traffic at unpredictable moments. The default assumption of almost every storefront platform, including the one this course uses
B2B	A business sells to another business	Few large orders, negotiated and customer-specific prices, credit terms rather than immediate payment, purchase orders, approval chains, and quantity breaks. Almost everything in Chapter ?? changes, and most of Chapter ??
C2C	A consumer sells to another consumer	The operator holds no stock and never owns the goods. The hard parts move to trust, dispute resolution and holding money in escrow — Chapter 3 is about this
C2B	A consumer sells to a business	Freelance marketplaces, stock photography, influencer deals. The “product” is a person’s time or licence, which breaks the stock model entirely

💡 Why the B2B/B2C split is the one that bites

A platform built for B2C assumes one price per product. B2B does not have one price per product — it has a price per product *per customer*, often negotiated, often with quantity breaks, sometimes expiring.

This is the most common way a student project, or a real small business, picks the wrong tool. Everything looks fine until the first wholesale customer asks for their agreed rate, and the answer turns out to require a different data model rather than a setting. Chapter 4 makes you ask this question before you choose, which is the only time asking it is cheap.

2.3 The Revenue Models

There are not very many, and most businesses combine two or three.



Figure 2.1: The five revenue models a shop can use, and the single fact about each that most shapes the software. Most real businesses run two of them at once.

2.3.1 Sales margin

You buy at one price and sell at another. The oldest model, and the one the storefront software in this course is built around by default.

Its distinguishing feature, from a software point of view, is that *you hold stock*. Stock means a quantity that can be wrong, can be reserved, can go negative under concurrency, and must be reconciled against physical reality. Chapter 6 and Chapter ?? spend real effort on this.

2.3.2 Commission

Somebody else sells; you take a percentage for having made it possible. This is the marketplace model, and it is attractive because you hold no stock and your costs barely rise with volume.

What you take on instead is everything in Chapter 3: two parties who do not trust each other, money that must be held until both are satisfied, and disputes that someone has to adjudicate. You have also acquired a problem the sales model does not have — you must attract *both* sides, and neither arrives before the other.

2.3.3 Subscription

The customer pays regularly for continued access or delivery. Revenue becomes predictable, which is why investors like it and why so much of the web has moved to it.

The software consequence is recurring billing, which is substantially harder than one-off payment: cards expire, payments fail silently, customers pause and resume, and the price may change mid-contract. Chapter ?? explains why a stored card is a different kind of integration from a single charge.

2.3.4 Advertising and affiliate

Advertising sells the attention of your visitors to somebody else. It needs large traffic before it pays anything at all, which makes it a poor first revenue model and a common fantasy.

Affiliate revenue is the mirror image: you send a visitor to somebody else's shop and are paid if they buy. Cheap to start, and it leaves you without a customer — you never learn who bought, cannot serve them again, and the relationship belongs to the merchant. Chapter ?? is where this becomes a strategy rather than an accident.

2.4 The Arithmetic of One Order

A revenue model is a claim that can be checked, and the check is simpler than students expect. Take one order and follow the money out of it.

Worked Example: Does this order make money?

A shop sells a pair of shoes at **Rs. 4,500**, delivered anywhere in Pakistan, with free returns. Work out what is left.

Step 1 — what came in.

Revenue per order: **Rs. 4,500**.

Step 2 — the cost of the thing itself.

The shoes cost the shop Rs. 2,800 to buy.

Remaining: $4,500 - 2,800 = \text{Rs. } 1,700$ (a gross margin of 38%).

Step 3 — the cost of taking the money.

The gateway charges 2.5% plus Rs. 15 per transaction: $0.025 \times 4,500 + 15 = 112.5 + 15 = \text{Rs. } 127.50$.

Remaining: $1,700 - 127.50 = \text{Rs. } 1,572.50$.

Step 4 — the cost of delivery.

Courier, Rs. 250, absorbed by the shop because delivery was advertised free.

Remaining: $1,572.50 - 250 = \text{Rs. } 1,322.50$.

Step 5 — returns, which are not optional.

20% of shoe orders come back. A return costs the courier fee again (Rs. 250) and the item cannot always be resold; assume a further Rs. 200 of loss on average. Expected cost per order placed: $0.20 \times (250 + 200) = \text{Rs. } 90$.

Remaining: $1,322.50 - 90 = \text{Rs. } 1,232.50$.

Step 6 — the cost of finding the customer.

The shop spends Rs. 60,000 a month on advertising and gets 50 orders from it: a **customer acquisition cost** of $60,000/50 = \text{Rs. } 1,200$.

Remaining: $1,232.50 - 1,200 = \text{Rs. } 32.50$.

The answer. The order that looked like a 38% margin earns **Rs. 32.50**, which is **0.7%** of the sale price — and that is before rent, salaries, hosting or tax. One extra return, or a 3% gateway instead of 2.5%, and it is a loss.

What this is really telling you. The two numbers that decided it were *returns* and *acquisition cost*, and neither appears anywhere in the product's price. A shop that measures only gross margin believes it is making 38%. This is why Chapter ?? is careful about which numbers on a dashboard are worth acting on.

☰ Contribution Margin, CAC and LTV

Contribution margin is what one order contributes after all the costs that vary with that order — goods, payment fees, delivery, expected returns. It is what is left to pay the costs that do not vary.

Customer acquisition cost (CAC) is the marketing spend divided by the customers it produced.

Lifetime value (LTV) is the total contribution margin a customer produces before they stop buying.

The test a business must pass is $LTV > CAC$, by a comfortable multiple. The worked example above passes only if the customer orders again — its CAC is nearly its whole first-order margin, which is survivable for a business with repeat customers and fatal for one without.

⚠ Where the network effect comes from, and where it does not

A **network effect** exists when the service gets better for each user as more users join. A marketplace has one: more sellers make it more useful to buyers, which attracts more sellers. A shop selling its own goods has *no* network effect. Your thousandth customer makes the shop no better for your first. Volume may buy you better purchase prices, which is a real advantage, but it is not a network effect and it does not protect you the same way.

This matters because the two justify completely different strategies. Running at a loss to grow is rational when a network effect will eventually make you defensible, and is just losing money when it will not. Plenty of ventures have adopted the first strategy while having the second business.

❗ Common Pitfalls

- **Naming a business model and thinking you have named a revenue model.** “We are a marketplace for tutors” says nothing about who pays.

- **Quoting gross margin as if it were profit.** The worked example went from 38% to 0.7% without anything unusual happening.
- **Forgetting returns.** In clothing and footwear they run at 20–40% and they are the difference between a viable shop and a dead one.
- **Advertising as a first revenue model.** It pays at scale and nothing before it, so a new venture choosing it has chosen to earn nothing for a long time.
- **Assuming the platform is model-agnostic.** It is not. A B2C storefront has one price per product, and discovering that during implementation is expensive — see Chapter 4.
- **Treating “free delivery” as marketing.** It is a cost of Rs. 250 per order that appears in no price list and is paid out of the margin.

Try It Yourself: The model behind the shop you will build

Still no software — Chapter 5 starts that. This laboratory produces the one-page brief that Chapter 4 turns into a decision and Chapter ?? is eventually marked against, so it is worth real effort.

Choose a shop to build. It must be plausible, small, and local enough that you can check your assumptions: a bookshop, a bakery, a tailor, a spare-parts dealer, a plant nursery. Do not choose “an Amazon competitor”.

Then produce, on one page:

1. **The business model in two sentences.** Who is the customer, what problem is solved, and why you rather than the shop down the road or the marketplace they already use.
2. **The classification:** B2C, B2B, C2C or C2B — and if you are tempted to say “both”, say which one the *software* must assume, because it can only assume one.
3. **The revenue model**, from Figure 2.1, with the reason for choosing it over the next most plausible one.
4. **The arithmetic of one order**, following the six steps of the worked example with your own figures. Find real numbers where you can: gateway fees and courier rates are published, and a shopkeeper will usually tell you their return rate if you ask politely. Mark every number you guessed.
5. **The two numbers that decide it.** From your own arithmetic, name the two inputs the result is most sensitive to, and say what would have to be true for the shop to lose money.

What to notice. Almost everyone finds that the contribution margin is far thinner than they expected, and that the deciding numbers are not the ones they thought they were building a business around. That discovery is the point of the exercise, and it is much cheaper to make here than in Chapter 23.

Checkpoint — Test Yourself

1. A food delivery app charges restaurants 20% commission, charges customers a delivery fee, and sells promoted placement in search results. Name the business model and all the revenue models, and say which single one you would expect to be most profitable per unit.
2. A shop has a contribution margin of Rs. 400 per order and a customer acquisition cost of Rs. 1,500. Under what condition is this business viable? State it as an inequality and say what would need measuring.
3. Explain why a B2B shop cannot usually be built by configuring a B2C platform, naming the specific assumption that breaks.

 Chapter Summary

- A business model says what value is created and for whom; a revenue model says who pays. They are independent, and a venture can have a good one and a fatal other.
- B2C, B2B, C2C and C2B classify by who transacts. The split that most affects the software is B2B against B2C: B2B has a price per customer, not a price per product.
- The five revenue models are sales margin, commission, subscription, advertising and affiliate. Holding stock or not is the fact about each that most shapes the system you have to build.
- Follow one order through six deductions — goods, payment fee, delivery, returns, acquisition — and the margin usually collapses. In the worked example, 38% became 0.7%.
- The viability test is $LTV > CAC$ by a comfortable multiple, which for a thin first-order margin means repeat custom is not optional.
- Marketplaces have network effects; shops selling their own goods do not, and strategies that assume one are expensive when it is absent.

 Review Questions

1. (MCQ) “We connect home cooks with office workers who want lunch.” This is: (a) a revenue model; (b) a business model; (c) both; (d) neither.
2. (MCQ) Which revenue model most changes the *payment* integration a shop needs? (a) Sales margin; (b) affiliate; (c) subscription; (d) advertising.
3. (Short) Distinguish gross margin from contribution margin, and give two costs that appear in one and not the other.
4. (Short) Explain why a marketplace has a harder launch than a shop, despite holding no stock.
5. (Short) A clothing shop reports 45% gross margin and is losing money. Give the two most likely explanations, and say what you would measure first.
6. (Applied) A shop sells a product at Rs. 2,000 that costs it Rs. 1,100. Gateway: 2.9% + Rs. 20. Delivery: Rs. 200, charged to the customer at Rs. 150. Returns: 12%, each costing Rs. 200 in courier and Rs. 150 in unsellable stock. Compute the contribution margin per order, showing each step. Then find the return rate at which the shop breaks even.
7. (Applied) Take the venture from this chapter’s laboratory and argue for a *different* revenue model than the one you chose. Give the strongest case you can, then say which software decisions in Parts III and IV would have to change if you adopted it.

Auctions, Portals and Communities

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — E-Commerce and Its Models

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the four common auction formats and **explain** what each one does to a bidder's incentive to bid honestly.
- **Identify** the engineering problems an auction creates that a fixed-price shop does not have.
- **Explain** the role of escrow, and **describe** what a marketplace operator must do when two users disagree.
- **Distinguish** a portal, a marketplace and a community, and **state** where each one's revenue comes from.
- **Explain** the two-sided launch problem and **evaluate** strategies for escaping it.

🔗 Before You Start

Chapter 2: the revenue models, especially commission, and the note on network effects at the end.

📖 Key Terms in This Chapter

auction • English auction • Dutch auction • sealed-bid auction • Vickrey auction • reserve price • proxy bidding • sniping • shill bidding • escrow • dispute resolution • reputation system • portal • marketplace • two-sided market • chicken-and-egg problem

3.1 Why This Chapter Is Here Rather Than at the End

The published outline lists “Auctions, Portals, and Communities” second from last, after security and deployment. That is where it appears in the book whose contents page the outline was copied from, and it is the wrong place: these are not an afterthought to building a shop, they are *alternative shapes* a business can take, and the decision between them belongs in Part I alongside the other models.

So this chapter sits where it belongs. It is also the chapter that explains why the rest of the book assumes a fixed-price shop: the alternatives are genuinely harder, and it is worth knowing exactly how.

3.2 Letting the Buyer Set the Price

A fixed-price shop answers “what is this worth?” before the customer arrives. An auction refuses to answer and lets the buyers decide.

That is attractive when the seller genuinely does not know the price — a second-hand item, a one-off, something whose value depends on who wants it. It is unnecessary when the seller does know, which is why most commerce is not conducted by auction.

Four Auction Formats

English auction — ascending, open. The price rises until one bidder remains. What nearly everyone means by “auction”.

Dutch auction — descending, open. The price starts high and falls until somebody accepts. Fast, and used for perishable goods where speed matters more than the last rupee.

Sealed-bid first-price — everyone submits one hidden bid; the highest wins and pays what they bid. Standard for tenders and contracts.

Vickrey auction (sealed-bid second-price) — the highest bid wins but pays the *second*-highest price. Its remarkable property is that bidding your true valuation is the best strategy whatever anyone else does, so there is nothing to be gained from strategising.

Why the second-price rule removes the guessing

Suppose an item is worth Rs. 10,000 to you.

In a *first-price* auction, bidding Rs. 10,000 guarantees you gain nothing even if you win — so you must shade your bid downward, and how far depends on guessing what others will do. Everyone does this, and everyone guesses.

In a *second-price* auction, bid Rs. 10,000. If the next highest bid is Rs. 8,000, you win and pay Rs. 8,000, gaining Rs. 2,000. If it is Rs. 11,000, you lose — and you are glad, because winning would have cost more than the item was worth to you. Raising your bid above your valuation can only make you win auctions you did not want; lowering it can only make you lose auctions you did want. Telling the truth is simply best.

Where you have met this already. The “proxy bidding” on consumer auction sites — you enter a maximum and the site bids on your behalf up to it — is a Vickrey auction wearing an English auction’s clothes. The winner pays one increment above the second-highest maximum, not their own.

3.3 What an Auction Costs You in Engineering

A fixed-price product is a row that changes rarely. An auction is a small real-time system, and it brings four problems the rest of this book does not have to solve.

Problem	Why a fixed-price shop does not have it
Time is part of the state	A listing must close at an exact moment, unattended, and settle correctly whether or not anyone is looking. A product page has no deadline. This needs a scheduler, and a scheduler that did not run is a silent failure
Concurrent bids	Two bids arriving in the same instant must produce one consistent outcome. This is the classic lost-update problem and it needs a transaction and a lock, not a quick read-then-write
Sniping	A bid placed one second before closing wins without giving anyone a chance to respond. Fixes exist — extend the closing time on a late bid — and each changes the rules of the auction, which is a design decision rather than a bug fix
Shill bidding	The seller, or a friend, bids to push the price up with no intention of buying. Detecting it means analysing bidding patterns across accounts, and it is fraud rather than a feature gap

⚠ The lost update, concretely

Two bidders submit Rs. 5,000 at the same instant on an item currently at Rs. 4,500.

A naive implementation reads the current high bid, checks that the new bid beats it, and writes. Interleaved, both read Rs. 4,500, both pass the check, and both write Rs. 5,000 — the second overwriting the first. One bidder's money is now owed on an item they did not win, and the audit trail shows a single bid.

The fix is the one Chapter ?? uses for stock: do the check and the write in one atomic statement, inside a transaction, with the row locked. “Read, decide, write” is three steps, and anything can happen between them.

3.4 Holding Other People's Money

An auction site that merely introduces a buyer to a seller has introduced two strangers and left. Whoever moves first — sends the money, or sends the goods — carries all the risk.

Escrow is the operator's answer: it holds the buyer's money, tells the seller to ship, and releases the money when the buyer confirms receipt or a time limit expires.

📄 Escrow and Dispute Resolution

Escrow is the holding of a payment by a trusted third party until the conditions of the transaction are met.

It makes the operator a custodian of money that is not theirs, which in most jurisdictions — Pakistan included — carries regulatory obligations (Chapter ??). It also makes the operator the **dispute resolution** authority, because somebody has to decide who is telling the truth when the buyer says the item never arrived and the seller says it did.

That adjudication cannot be automated away, and it is why marketplace commissions are higher than payment processing fees: the commission buys arbitration, not just plumbing.

🔗 Reputation as the cheap substitute, and what it cannot do

Escrow is expensive. The cheaper substitute is a **reputation system**: record how every past transaction went and let future counterparties read it.

It works, within limits, and the limits are worth knowing:

- **A new seller has no reputation**, so the system gives no protection in exactly the case where it is most needed.
- **Reputation can be bought** — a hundred small honest sales, then one large dishonest one. The score is an average; the fraud is a maximum.
- **Retaliation distorts it**. If each side rates the other, a buyer who reports a problem risks a bad rating in return, so both sides learn to stay quiet and the scores drift upward until they mean nothing.
- **It cannot be transferred**. Years of standing on one platform are worth nothing on another, which is exactly why the platform likes it.

3.5 Portals, Marketplaces and Communities

The three are often lumped together and should not be.

A **portal** aggregates *information* and sells attention — a price comparison site, a news aggregator with shopping links. It carries no stock, handles no payment, and earns from advertising or affiliate referral. Its engineering problem is ingesting other people's data and keeping it fresh, and its

commercial weakness is that it owns no transaction and can be cut out the moment a supplier decides to stop feeding it.

A **marketplace** aggregates *sellers* and hosts the transaction. It earns commission, and it is a shop in which the catalogue is written by strangers — which turns catalogue quality, duplicate detection and fraud into daily operational work rather than a one-time setup.

A **community** aggregates *people* who are there for each other rather than to transact. Commerce, if any, is secondary, and the usual mistake is assuming an engaged community will convert into a profitable marketplace. Many do not: the thing people valued was the absence of selling.

💡 The two-sided launch problem

A marketplace is worthless to buyers with no sellers, and worthless to sellers with no buyers. On day one it has neither, and every new one faces this.

The strategies that actually work are all variations on *refusing to be two-sided at first*:

- **Be one of the sellers yourself** until there are enough others. You take on stock you did not want, and you compete with your own sellers, and it works.
- **Pick one narrow side and own it completely** — a single city, a single category — so that the market is small enough to fill.
- **Be useful to one side alone.** Offer sellers a tool worth having even with no buyers (stock management, a storefront), then introduce buyers once the sellers are already there.
- **Fake the thin side** by listing supply gathered from elsewhere. Common, legally risky, and the point at which several well-known companies acquired their reputations.

Notice that none of these is “build a better marketplace”. The problem is not a product problem, and a student project that proposes a marketplace without addressing it has not yet met its hardest part.

⚠ Common Pitfalls

- **Implementing bidding with read-then-write.** Two simultaneous bids will eventually produce one lost bid, and it will be unreproducible when reported.
- **Closing auctions with a page request.** If closing happens when somebody loads the page, an auction nobody looks at never closes. It needs a scheduled job, and the job needs monitoring.
- **Treating escrow as a feature.** It is custody of other people’s money and it is regulated. Chapter ?? is not optional reading if you intend to hold funds.
- **Believing a reputation score protects the first transaction.** It protects the hundredth.
- **Proposing a marketplace without a plan for the empty side.** This is the single most common fatal flaw in student project proposals for this course.
- **Assuming a community will monetise.** The engagement you measured may exist precisely because nothing was being sold.

🔧 Try It Yourself: Three shapes, and what each one must get right

No software yet. This is the last of the three analysis laboratories; Chapter 5 starts building. Find one live example of each: an **auction**, a **marketplace** and a **portal**. They may be Pakistani or international, but you must be able to browse all three.

For each, record:

1. **Who takes the risk** if the goods never arrive — buyer, seller or operator? Find the actual policy page, do not guess. Quote it.

2. **Where the money is** between payment and delivery. Is there escrow? How long is it held, and what releases it?
3. **What the operator earns**, and from which side.
4. **One rule that exists because of a specific abuse.** Every mature platform has several — a bidding increment, a minimum feedback score, a holding period for new sellers, a limit on listing edits. Name one and work out what it is defending against.

Then, for the **auction** only: find the closing rules. Does a late bid extend the auction? By how much? Say which auction format that makes it, and whether it rewards bidding your true valuation.

Write up half a page ending with one sentence: of the three, which would be hardest to build, and which single problem makes it so?

What to notice. The rules that look arbitrary are nearly always scar tissue. Each one is a record of something that went wrong, and reading a mature platform's rules is the cheapest way to learn what will go wrong with yours.

🔍 Checkpoint — Test Yourself

1. In a Vickrey auction an item is worth Rs. 15,000 to you. Bids of Rs. 12,000 and Rs. 13,500 have been placed by others. What should you bid, what do you pay if you win, and why would bidding Rs. 18,000 be a mistake?
2. An auction site closes listings when a visitor loads the listing page. Describe precisely what happens to an auction for an unpopular item, and what the seller sees.
3. A marketplace has plenty of buyers and too few sellers. Give two strategies from this chapter that would address it, and name a cost of each.

📋 Chapter Summary

- Auctions, portals, marketplaces and communities are alternative shapes a business can take, which is why this chapter sits in Part I rather than where the published outline puts it.
- The four formats are English, Dutch, sealed-bid first-price and Vickrey. Under the Vickrey rule, bidding your true valuation is optimal whatever others do — and consumer proxy bidding is Vickrey in disguise.
- An auction adds four problems a fixed-price shop does not have: time as state, concurrent bids, sniping and shill bidding. The first two are engineering; the second two are rule design.
- “Read the high bid, check it, write it” loses bids under concurrency. The check and the write must be one atomic operation.
- Escrow makes the operator a custodian of other people's money and the arbiter of their disputes. Reputation is the cheap substitute and fails exactly on the first transaction and the large one.
- Portals aggregate information, marketplaces aggregate sellers, communities aggregate people. Only the marketplace owns the transaction.
- Every two-sided market launches empty, and every strategy that works amounts to refusing to be two-sided at the start.

 Review Questions

1. **(MCQ)** In a second-price sealed-bid auction, the optimal strategy is to bid: (a) slightly above your valuation; (b) exactly your valuation; (c) slightly below it; (d) it depends on the other bidders.
2. **(MCQ)** Which problem does a fixed-price shop share with an auction site? (a) Sniping; (b) shill bidding; (c) concurrent updates to one row; (d) scheduled closing.
3. **(Short)** Explain sniping, and give one rule change that discourages it together with the cost of that change.
4. **(Short)** Distinguish a portal from a marketplace, and say which is more easily cut out of its own business.
5. **(Short)** Give three distinct ways a reputation system can be gamed or rendered meaningless.
6. **(Applied)** Design the database handling for a single bid. Give the table columns you would need, the exact check that must happen atomically, and state what goes wrong if the check and the write are separate statements. You may write it as SQL or as precise prose.
7. **(Applied)** A student proposes a marketplace connecting Bahawalpur tailors with customers. Identify the two-sided launch problem in this specific case, choose one strategy from this chapter, and describe concretely what the first eight weeks would consist of.

Planning an E-Commerce Framework

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — E-Commerce and Its Models

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** the four ways a shop can be obtained, and **state** what each costs in money, time and control.
- **Write** requirements for a shop in a form that discriminates between those four rather than describing all of them equally.
- **Identify** the small number of requirements that actually decide the choice, and **explain** why most requirements do not.
- **Compute** a total cost of ownership over three years, including the costs that do not appear on a price list.
- **Defend** a build-buy-or-extend decision against the obvious objection to it.

🔗 Before You Start

Chapter 2 for the business and revenue model, and its laboratory — this chapter turns that one-page brief into a decision. Chapter 3 if your venture is a marketplace rather than a shop.

📖 Key Terms in This Chapter

requirement • functional requirement • non-functional requirement • SaaS • hosted platform • self-hosted • open source • bespoke development • total cost of ownership • vendor lock-in • extensibility • data portability • decision matrix • minimum viable product

4.1 The Question This Chapter Answers

You have a business model, a revenue model and some idea of what you are selling. There is now exactly one decision to make before any of the rest of this book applies, and it is this: *where does the software come from?*

There are four answers, and the whole of Parts II to V is a consequence of which one you pick.

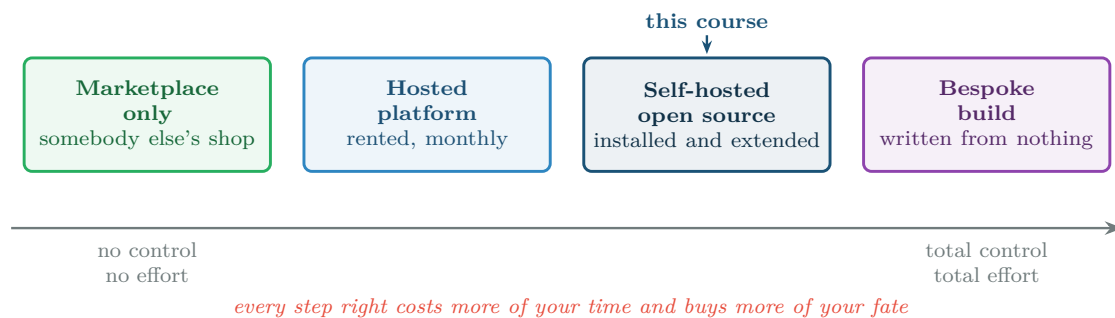


Figure 4.1: The four ways to obtain a shop. They are a spectrum, not a menu: each step to the right buys control and costs effort, and the right answer depends on which of the two you are shorter of.

4.2 The Four Options, Honestly

Option	What it is good at	What it costs you
Marketplace only	Customers on day one. No software, no hosting, no payment integration. Genuinely the right answer for many small sellers	You perform none of the four functions (Chapter 1). You do not own the customer relationship, cannot email them, and can be delisted without appeal. The commission is the visible cost and the smallest one
Hosted platform	Running in an afternoon, someone else patches it, payment and shipping already integrated	A monthly fee that scales with your success, a transaction cut on top, and a hard ceiling: when the platform cannot do something, it cannot do it. Getting your data out ranges from awkward to impossible
Self-hosted open source	You can change anything, including the things the vendor never anticipated. No per-sale cut. The data is in your database	You are now responsible for hosting, backups, updates and <i>security</i> — an unpatched shop is a liability, not an inconvenience. Needs someone who can do Chapter ??
Bespoke build	Fits the business exactly, because it was built for it. The only option when the business is genuinely unusual	Twelve to eighteen months before you sell anything, and you will reimplement baskets, tax tables and payment retries that have been solved for twenty years — badly, because you will meet each edge case for the first time in production

⚠ The outline this course follows assumes the fourth option

The HEC outline for this course is, clause for clause, the contents page of a 2010 book about writing a shopping cart from scratch in PHP 5 — see the Course Specification. Taken literally, it asks a 5th-semester class to choose *bespoke build*.

That was defensible in 2010 and is not now, for a reason this chapter can state precisely: the work has moved. In 2010, writing a basket and a checkout was most of the job. Today a mature, free, auditable implementation of both is one command away, and the job that remains — modelling the catalogue, configuring tax and shipping, integrating payment, securing the result, extending what the platform cannot do — is both larger and more representative of what an IT graduate is actually hired to do.

So this course teaches every *topic* the outline lists, and reaches them by the third route rather than the fourth. You should be able to defend that choice, not merely inherit it, which is what this chapter is for — and Chapter ?? is where the “extend” in “self-hosted and extend” stops being a word and becomes code you write.

4.3 Requirements That Actually Discriminate

Students write requirements like “the system shall allow customers to add products to a basket”. Every one of the four options does that. A requirement that all four satisfy carries no information and cannot help you choose.

Functional and Non-Functional Requirements

A **functional requirement** says what the system must *do*: accept a voucher code, calculate tax by province, email an invoice.

A **non-functional requirement** says how well, under what constraints: how fast, how many at once, how available, how secure, how maintainable, by whom.

For the build-buy-extend decision the useful requirements are nearly all non-functional or unusual-functional. The ordinary functional ones are table stakes — everything does them — and listing them at length is the most common way a requirements document ends up being thirty pages that decide nothing.

The five questions that decide it

In practice the choice turns on a handful of questions. Ask these first, and most of the rest is detail.

1. **Is there anything genuinely unusual about how you price, sell or deliver?** Per-customer negotiated pricing, made-to-measure items, rental with a return date, services booked by time slot. If yes, every option except the last two gets harder fast.
2. **Who will maintain it in two years?** Not who will build it. A self-hosted shop with nobody to patch it becomes a breach (Chapter ??). If the honest answer is nobody, choose a hosted platform and accept the ceiling.
3. **What volume, and how uneven?** Thirty orders a day and three thousand are different systems; so are steady traffic and traffic that multiplies tenfold for one week a year.
4. **How much does leaving cost?** Can you export your products, customers and order history in a usable form? Ask it on day one, because the answer never improves.
5. **What is the budget, over three years, including somebody’s time?** Not the setup cost. See below.

4.4 The Cost That Is Not on the Price List

The commonest error here is comparing a monthly fee against “free” and concluding that open source is cheaper. Both numbers are wrong.

Worked Example: Three years of a small shop, two ways

A shop expecting **Rs. 800,000 of sales a month**, growing slowly. Compare a hosted platform against self-hosted open source over three years.

Hosted platform.

- Subscription: \$79/month $\approx$ Rs. 22,000. Over 36 months: **Rs. 792,000.**
- Platform transaction fee, 0.5% of sales — charged on top of the gateway’s own fee: $0.005 \times 800,000 \times 36 =$ **Rs. 144,000.**
- Two paid add-ons at \$15/month each $\approx$ Rs. 8,400/month: **Rs. 302,400.**
- Theme and setup, one-off: **Rs. 60,000.**
- Maintenance labour: near zero; the vendor patches.
- **Total: Rs. 1,298,400.**

Self-hosted open source.

- Licence: **Rs. 0**. This is the number people stop at.
- Hosting, a modest virtual server plus backups, Rs. 9,000/month: **Rs. 324,000**.
- Setup and theming, 70 hours of somebody at Rs. 2,500/hour: **Rs. 175,000**.
- Two custom modules the platform does not ship, 40 hours: **Rs. 100,000**.
- **Maintenance: 4 hours a month** — updates, patches, backups checked, the occasional breakage. $4 \times 36 \times 2,500 = \text{Rs. 360,000}$.
- **Total: Rs. 959,000**.

The result. Self-hosting wins by Rs. 339,400 over three years — about 26%. But look at what decided it: **maintenance labour is Rs. 360,000**, the single largest line in the self-hosted column and the one most often entered as zero. Set it to zero and self-hosting appears to win by 87%. Double it, to eight hours a month because nobody on staff really knows the platform, and the two options are level.

The lesson. This calculation is not really about software prices. It is a question about whether you have, and will keep, somebody who can maintain a server — and if the answer is no, the hosted option is cheaper at any subscription price, because the alternative is not a cheaper shop but an unpatched one.

⚠ Two costs that appear in neither column

Vendor lock-in is not a line item until the day you want to leave, at which point it is the whole bill. Before committing, actually export your catalogue and customers from a trial account and look at what comes out. “We support CSV export” and “you can reconstruct your shop from this” are different claims.

The ceiling has no price at all. It is the feature you cannot have, discovered in month fourteen, when the business depends on it. The hosted option’s real cost is the probability that this happens multiplied by what it does to you — which is why question 1 above is first.

4.5 Making the Decision Defensible

Write it down as a matrix: criteria in rows, weights, options scored. Not because the arithmetic is wise — it is easy to fit the weights to the answer you wanted — but because it forces the weights into the open where somebody can disagree with them.

🔗 A decision matrix for the shop in this course

Criteria weighted out of 100, each option scored 1–5.

Criterion	Weight	Hosted	Self-host	Bespoke
Students must see and change real code	30	1	5	5
Working shop within three weeks	25	5	4	1
Zero licence or subscription cost	15	2	5	5
Runs identically on every laptop	15	3	5	4
Covers the HEC clause list directly	15	3	5	4

Weighted total	100	2.75	4.75	3.70
----------------	-----	------	------	------

Read the weights, not the totals. The first row is worth 30 and it is where hosted loses the contest — a course whose students cannot read the code is not this course. Change that weight to 5, as a small shopkeeper would, and hosted wins outright. The matrix did not make the decision; it made the decision *visible*, which is all a matrix is for.

! Common Pitfalls

- **Requirements that every option satisfies.** “Customers can add to a basket” discriminates between nothing.
- **Entering maintenance as zero.** It was the largest single line in the worked example, and leaving it out reverses the answer.
- **Comparing setup cost instead of three-year cost.** Subscriptions and labour both accumulate; neither appears in a setup quote.
- **Choosing bespoke because the platform is “bloated”.** You will reimplement the parts you called bloat, in production, one incident at a time.
- **Choosing self-hosted with nobody to maintain it.** This is not a cheaper shop. It is a shop that will be compromised, and Chapter ?? shows how quickly.
- **Not testing the export before committing.** Lock-in costs nothing until it costs everything.
- **Weighting the matrix after seeing the totals.** Everyone is tempted; write the weights down first and date them.

🔧 Try It Yourself: Your requirements, and your decision

The last analysis laboratory. Chapter 5 installs software.

Using the venture from Chapter 2’s laboratory, produce a two-page document:

1. **Twelve requirements**, each labelled functional or non-functional, each numbered so you can refer to it later. At least four must be non-functional, with a number attached: not “must be fast” but “a category page must render in under one second with 5,000 products”. You will test some of these in Chapter 6 and Chapter ??, so make them checkable.
2. **Mark the discriminating ones.** Go through your twelve and strike out every requirement that all four options satisfy. Most students strike out eight or nine. What is left is your real requirements document.
3. **Answer the five questions** from this chapter, in writing, including the honest answer to “who maintains this in two years?”
4. **A three-year total cost of ownership**, two columns, following the worked example. Mark every figure you guessed, and state the one number the answer is most sensitive to.
5. **A decision matrix**, weights written before scores, and a two-sentence conclusion.
6. **The strongest objection to your decision**, and your answer to it. If you cannot state a real objection, you have not understood the option you rejected.

What to notice. Your decision does not have to be the one this course makes. A student whose venture is three products sold through Instagram should conclude “marketplace only” and say so — and will still build the shop in Parts II–V, because the course has to teach the general case. Being right about your own venture and building something else deliberately is a perfectly coherent position, and a more useful one than pretending.

? Checkpoint — Test Yourself

1. Classify each as functional or non-functional, and say which of them discriminates between the four options: (a) customers can reset a forgotten password; (b) prices are agreed per customer and may differ for the same product; (c) the shop stays up during a sale that brings twenty times normal traffic.
2. A shop compares Rs. 22,000/month hosted against “free” open source and concludes open source saves Rs. 792,000 over three years. Name the three largest costs they have omitted.
3. Give a specific business for which this course’s choice — self-hosted open source — would be the wrong answer, and say which option you would choose instead and why.

✓ Chapter Summary

- A shop can be obtained four ways: marketplace only, hosted platform, self-hosted open source, or bespoke build. They form a spectrum trading control against effort.
- Requirements that every option satisfies cannot help you choose. Strike them out; what remains is the document that matters.
- Five questions decide most cases: anything unusual about pricing or delivery, who maintains it in two years, volume and its unevenness, the cost of leaving, and three-year budget including labour.
- Compare total cost of ownership over three years, not setup cost. In the worked example maintenance labour was the largest self-hosted line, and entering it as zero reverses the conclusion.
- Vendor lock-in and the feature ceiling appear in no price list and are often the real cost of the hosted option.
- A decision matrix does not make the decision; it makes the weights visible so somebody can argue with them. Write the weights first.
- This course chooses self-hosted open source and teaches every clause of the outline through it, rather than writing a cart from scratch as the 2010 book behind the outline assumed. You should be able to defend that, including knowing when it is the wrong answer.

✍ Review Questions

1. **(MCQ)** Which is a non-functional requirement? (a) The shop emails an invoice after each order; (b) the checkout completes in under two seconds at 200 concurrent users; (c) customers can apply a voucher code; (d) administrators can issue refunds.
2. **(MCQ)** In the worked example, the largest single cost in the self-hosted column was: (a) hosting; (b) setup and theming; (c) custom modules; (d) maintenance labour.
3. **(Short)** Explain why a requirement satisfied by all four options is useless for this decision, and give an example of one.
4. **(Short)** What is vendor lock-in, and what concrete test would you run during a free trial to measure it?
5. **(Short)** Give the strongest argument *against* the choice this course makes, and then answer it.
6. **(Applied)** A tailoring business takes measurements in the shop, makes garments to order over two weeks, and prices each job individually after seeing the cloth. Work through the five questions and recommend one of the four options, justifying it. Identify the one requirement that makes this case different from a normal shop.

7. (**Applied**) Build a three-year total cost of ownership for a shop doing Rs. 3,000,000 a month — nearly four times the worked example — on both options, keeping all other assumptions. State which option wins, and identify the single assumption that, if changed, would reverse it.

PART II

Standing the Shop Up

From here on there is a shop. Part II brings it up, fills it with a catalogue that survives contact with real products, and makes it look like somebody's rather than nobody's — and it does all of that without installing a single thing on your own machine.

screens on top, tables underneath — both, or neither is understood

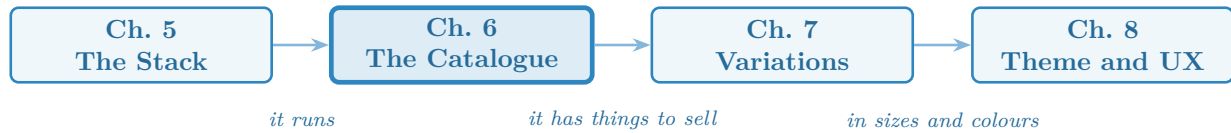


Figure P2.1: Reading path through Part II. Chapter 6 is the one to slow down on: it is the only chapter that looks at the same thing from the admin panel and from the database at once.

The Stack, and Your Shop in One Command

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Standing the Shop Up

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the four layers a web shop runs on and **state** what each one is responsible for.
- **Explain** what a container is, and **distinguish** it from a virtual machine and from an installer.
- **Read** a Compose file and **say** what each service, port and volume in it does.
- **Bring up** a working storefront, **verify** it from outside, and **reset** it to a known state.
- **Measure** what the stack costs to start and to store, and **distinguish** the cost paid once from the cost paid daily.

🔗 Before You Start

Chapter 4: the decision to use a self-hosted open-source platform, and why. Appendix A must be finished before this chapter — you need a working container runtime, and if it is going to fight you, it will fight you now rather than in Week 9.

📖 Key Terms in This Chapter

stack • LAMP • web server • application server • database server • container • image • volume • port mapping • Docker Compose • service • environment variable • reproducibility • idempotent • pinning • checksum

5.1 Four Layers, and What Each One Does

When you type a shop’s address into a browser, four distinct pieces of software answer in turn. Knowing which is which is the difference between fixing a problem and restarting things until it goes away.

💡 Which layer is this problem in?

A diagnostic habit worth building on day one, because it turns “the site is broken” into a question with an answer.

- **Nothing answers at all**, the browser cannot connect: the web server is down, or the port is wrong. Nothing above it has run.
- **A page appears but says “500”**: the web server is fine; the application failed. The reason is in the PHP error log, never on the screen — and Chapter ?? explains why you want it that way.
- **A page appears, looks right, but data is missing or stale**: the application ran and the database answered something unexpected.
- **Everything works and is slow**: usually the database, and Chapter 6 measures exactly that.

A slow page is usually this layer. A lost order is always this one.

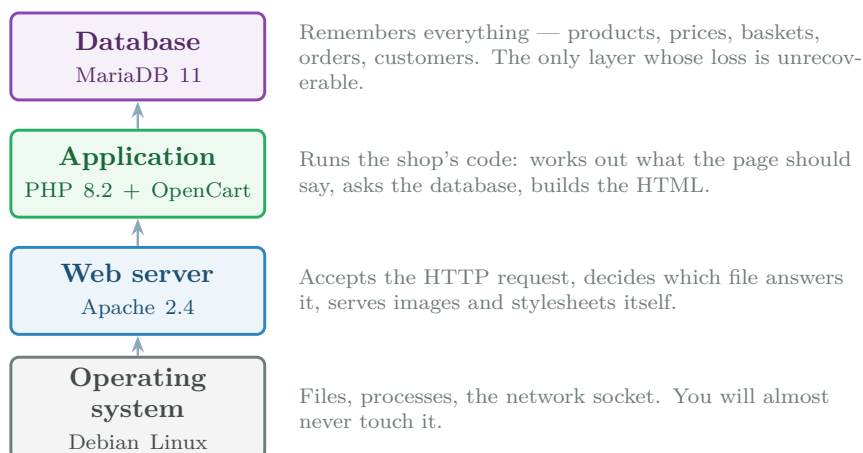


Figure 5.1: The four layers. The traditional name for this arrangement is LAMP — Linux, Apache, MySQL, PHP — and although every letter has alternatives now, the shape has not changed in twenty-five years.

5.2 Containers, and What They Are Not

The traditional way to get those four layers is to install them: a web server, a database, a PHP runtime, each with its own installer, its own configuration file in its own place, and its own way of being slightly different on your machine from on everybody else's.

A **container** replaces that with a package that already contains the software *and* everything it depends on, down to the system libraries, arranged exactly as its author arranged it.

Image, Container, Volume

An **image** is a read-only, layered filesystem containing an application and its dependencies. It is built once and is identical everywhere.

A **container** is a running instance of an image. It has its own filesystem view and its own network identity, but it shares the host's kernel — which is what makes it start in a second rather than a minute.

A **volume** is storage that lives *outside* the container and survives it. This is the one that matters: everything inside a container is thrown away when the container is removed, so anything you want to keep — your database, your shop — must be in a volume.

Three things a container is not

It is not a virtual machine. A VM carries a whole operating system including its own kernel, takes a minute to boot and gigabytes of memory. A container shares the host kernel and starts in about a second. The isolation is correspondingly weaker, which matters in production and not in this course.

It is not a backup. “It is in the container” means it will be gone when the container is replaced, which happens every time you rebuild. Your shop survives because it is in a *volume*, and Chapter ?? is about what happens when somebody has not understood that distinction.

It is not automatically secure. A container with a weak password is a weak password that starts quickly. Chapter ?? attacks this very stack and finds things.

5.3 The Three Services in This Course's Stack

The whole stack is described by one file, `code/docker-compose.yml`, and it declares three services.

```
services:
  db:                                     # MariaDB 11: remembers everything
    image: mariadb:11
    volumes:
      - dbdata:/var/lib/mysql           # the shop's memory, kept outside
    healthcheck: ...                    # "accepts a login", not "started"

  shop:                                  # Apache + PHP 8.2 + OpenCart 4.1.0.4
    build: ./shop
    depends_on:
      db: { condition: service_healthy }
    ports:
      - "8090:80"                       # your 8090 -> the container's 80
    volumes:
      - shopdata:/var/www/html

  adminer:                               # a database console in the browser
    image: adminer:latest
    ports:
      - "8091:8080"
```

Three decisions in that file are worth understanding, because each one is a mistake people make.

The healthcheck, not just depends_on. Ordering container *starts* is not enough. MariaDB accepts TCP connections several seconds before it will accept a login, so a shop that waits only for the database to have started will try to install itself against a database that refuses it — intermittently, and more often on a fast machine. The condition `service_healthy` waits for the database to say it is actually ready.

The port mapping is a translation, not a setting. `8090:80` means “traffic arriving at port 8090 on my machine goes to port 80 inside the container”. The shop always believes it is on port 80. If 8090 is busy on your machine — and on a university laptop it often is — change the left-hand number only.

Named volumes, so that a reset is deliberate. `docker compose down` stops everything and keeps your shop. `docker compose down -v` removes the volumes too, and the next start reinstalls from nothing. That second command is how you recover from a shop you have broken beyond understanding, and it is also how you destroy a semester’s work in week 14. It is worth knowing which is which before you need either.

💡 Why the image is built here rather than downloaded

Most teaching material says “pull the OpenCart image”. There is no longer one to pull: there has never been an official OpenCart image, and the widely-used third-party one was withdrawn. Every such instruction on the web is now dead.

So `code/shop/Dockerfile` builds it, from the official PHP image plus a *pinned* OpenCart release — pinned by version and by SHA-256 checksum, so that the shop you build this year is the shop a classmate builds next year. Two lines in that file do the real work:

- `ARG OC_VERSION=4.1.0.4` — the exact release.
- `echo "${OC_SHA256} /tmp/oc.zip | sha256sum -c -` — the download really is the file we tested against. A version number alone is not reproducible, because a release asset can be replaced.

There is a second reason, and it is the pedagogical one: Figure 5.1 is a diagram of something, and pulling one opaque image would have left you unable to point at any of it. Here the layers are in the file, in order.

⚠ OpenCart ships its own Dockerfile, and it does not work

The release archive contains a `Dockerfile` and a `docker-compose.yml` at the top level. This course uses neither, for two reasons you can verify yourself in five minutes.

The `Dockerfile` contains a shell syntax error — `docker-php-ext-install zip && && docker-php-ext-enable zip` — so it has never built. The compose file pins `mysql:5.7`, which reached end of life in October 2023 and receives no further security fixes, and starts Redis, Memcached and PostgreSQL that a stock install does not use.

The lesson is not that OpenCart is bad software; it is a mature project that runs a large share of the world's small shops. The lesson is that an artefact being official is evidence and not authority. You are allowed to check, and in this course you are expected to.

5.4 What Happens the First Time

One command brings the whole thing up:

```
docker compose up -d --build
```

On a first run this does rather a lot: downloads about 1.5 GB of images, builds the shop image, starts three containers, waits for the database, and then runs OpenCart's command-line installer, which creates 159 tables and populates them with a demonstration catalogue. On every later run it does almost none of it.

That difference is worth measuring rather than asserting.

📊 Measured: What the Stack Costs to Start

Produced by `code/m05_coldstart.py -scratch`, which removes every image and volume first, on the machine in Appendix A — 4 cores, 7.7 GiB available to the container runtime.

Start	To first 200	What it had to do
From scratch (no images at all)	338.4 s	Download 1.5 GB, compile three PHP extensions, fetch and verify OpenCart, install 159 tables. Of this, 214.6 s was the build alone
Cold (images built, no shop)	34.1 s	Start three containers, wait for the database, install OpenCart
Warm (the shop already exists)	16.4 s	Start three containers. Nothing else

Footprint. Images: shop 882 MB, MariaDB 462 MB, Adminer 164 MB — **1,508 MB** in all. Volumes after installation: database 181 MB, shop files 40 MB. About 1.7 GB of disk for a complete shop.

The ratio is the point. From scratch to warm is **20.6×**. You pay the 338 seconds once, in Week 3, and the 16 seconds perhaps sixty times over the semester. Quoting a single “startup time” for this stack would describe an experience nobody has.

On variance. An earlier run of the same measurement on the same machine gave 17.6 s cold and 7.7 s warm — roughly half — because less was running at the time. Your numbers will differ from these by more than you expect. The *ratio* between cold and warm should not.

5.5 Looking At What You Were Given

The installer leaves you a working shop with a demonstration catalogue. It is worth knowing its shape, because every chapter in Part II changes part of it.

🔗 The shop, counted

Asked of the database directly, through Adminer or the command line:

```
SELECT (SELECT COUNT(*) FROM oc_product) AS products,
       (SELECT COUNT(*) FROM oc_category) AS categories,
       (SELECT COUNT(*) FROM oc_order)   AS orders,
       (SELECT COUNT(*) FROM oc_customer) AS customers;
```

- **159 tables** in total.
- **19 products** and **38 categories** of demonstration data.
- **0 orders** and **0 customers** — nobody has shopped here yet.
- **18 tables whose name begins oc_product.** One product is not one row, and Chapter 6 is about why.

That last number is the one to sit with. A beginner expects a products table. There are eighteen, because a product has descriptions in several languages, images, options, attributes, discounts, related items and category memberships — and each of those is a different relationship.

! Common Pitfalls

- **Running down -v when you meant down.** The -v removes the volumes, which is your shop. There is no undo.
- **Editing files inside a running container.** They are gone at the next rebuild. Changes that must survive belong in the image or in a volume.
- **Assuming up means ready.** It means *started*. On a first run the shop answers about twenty seconds later; every measurement in this book calls `waitfor.py` first for exactly this reason.
- **Changing the right-hand side of a port mapping.** In 8090:80, the 80 is the port inside the container and the shop expects it. Change the left number.
- **Treating a container as a backup.** It is the opposite of one.
- **Giving up when it does not start.** Appendix A lists the five failures that account for nearly all of them, and four are a busy port or a runtime that is not actually running.

Try It Yourself: Your shop, up, verified, broken and restored

From here to Chapter ?? this is the same shop. Treat it accordingly.

```
# --- 1. Bring the whole stack up -----
# First run: expect about five minutes and 1.5 GB of download.
cd code
cp .env.example .env
docker compose up -d --build

# --- 2. Watch it install -----
# Not optional. You should see "SUCCESS! OpenCart successfully
# installed" go past; if you never see it, nothing below will work.
docker compose logs -f shop

# --- 3. Verify from outside, not from the logs -----
# Three URLs must answer 200. waitfor.py checks all three and is the
# answer to "is my shop up?" for the rest of the semester.
python waitfor.py

# --- 4. Look at the shop, as a customer and as the owner -----
# Storefront    http://localhost:8090/
# Admin         http://localhost:8090/admin/    (admin / admin123)
# Database      http://localhost:8091/          (shop / shoppw / opencart)

# --- 5. Count what you were given -----
# In Adminer, or here. 159 tables; 19 products; 0 orders.
docker compose exec db mariadb -ushop -pshoppw opencart \
    -e "SELECT COUNT(*) FROM information_schema.tables \
        WHERE table_schema='opencart';"

# --- 6. Prove the volume is doing what you think -----
# Change something in the admin panel first -- edit a product name.
# Then stop WITHOUT -v and start again. Your change is still there.
docker compose down
docker compose up -d
python waitfor.py

# --- 7. Now reset it deliberately -----
# -v throws the volumes away. The shop reinstalls from nothing and
# your edit is gone. Do this once, now, while it costs you nothing.
docker compose down -v
docker compose up -d
python waitfor.py

# --- 8. Measure it -----
# Produces the table quoted above, on YOUR machine.
python m05_coldstart.py --warm
```

What to notice. Step 6 and step 7 differ by two characters and by everything else. Having done both deliberately, in a week when the shop contains nothing you care about, is the cheapest possible way to learn the difference.

Also notice that step 3 does not trust step 2. The log said the install succeeded; step 3 asks the shop. Throughout this course, the thing that reports success and the thing that is true are kept apart on purpose.

Checkpoint — Test Yourself

1. A classmate's shop returns "500 Internal Server Error" on every page. Which of the four layers in Figure 5.1 are known to be working, which is known to have failed, and where would you look first?

2. Explain why `depends_on` alone is not enough to order the database before the shop, and describe the failure it produces.
3. You run `docker compose down -v` by mistake in Week 12. State exactly what is lost, what is not, and what would have had to exist beforehand for this to be a minor inconvenience.

Chapter Summary

- A shop runs on four layers — operating system, web server, application, database. Knowing which layer a symptom belongs to is most of diagnosis; only the database's loss is unrecoverable.
- An image is a built filesystem, a container is a running instance of one, and a volume is the only part that survives the container. A container is not a virtual machine, not a backup, and not secure by itself.
- The whole stack is three services in one Compose file: database, shop, database console.
- `depends_on` orders starts, not readiness. A healthcheck is what makes the shop wait until the database will actually accept a login.
- `down` keeps your shop; `down -v` destroys it. Learn the difference in Week 3.
- The image is built from a pinned release verified by checksum, because no maintained OpenCart image exists to pull and because a version number alone is not reproducible.
- Measured on this machine: 338.4 s from scratch, 34.1 s cold, 16.4 s warm — a ratio of 20.6× between the cost paid once and the cost paid daily. The stack occupies about 1.7 GB.
- The installed shop has 159 tables and 19 products, and eighteen of those tables are about products. One product is not one row.

Review Questions

1. (MCQ) In the port mapping 8090:80, the 80 is: (a) the port on your machine; (b) the port inside the container; (c) the database port; (d) an arbitrary label.
2. (MCQ) Which survives `docker compose down`? (a) Nothing; (b) the containers; (c) data written to a named volume; (d) files edited inside a running container.
3. (Short) Distinguish a container from a virtual machine, and give one consequence of the difference that matters in this course.
4. (Short) Why does this course build the shop image from a pinned release rather than pulling a ready-made one? Give both reasons.
5. (Short) A page loads but shows yesterday's price. Which layer is at fault, and what are the two most likely explanations?
6. (Applied) Your stack will not start: port 8090 is already in use. Describe exactly what you would change, in which file, and what you would then have to change in the shop's own configuration for links to keep working. (Look at `OC_HTTP_SERVER` before answering.)
7. (Applied) Design a measurement that would show whether the *database* or the *application* layer is responsible for a slow category page, using only tools in this chapter. State what you would record, at what catalogue sizes, and what result would point at each layer. Chapter 6 runs your experiment; write it first.

Products, Categories and Their Tables

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Standing the Shop Up

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why one product occupies eighteen tables, and **say** what decides which table a given fact belongs in.
- **Create** products and categories through the admin panel, and **find** in the database exactly what each screen wrote.
- **Describe** how a category hierarchy is stored, and **say** what the path table buys and what it costs.
- **Read** the query a category page runs, and **use** EXPLAIN to say what the database does with it.
- **Measure** how the catalogue's size affects the page, and **decide**, from evidence, whether an index would help.

🔗 Before You Start

Chapter 5: a running shop. *Database Systems* for joins, keys and indexes; Appendix B if those need refreshing. You will spend as much of this chapter in the database console as in the admin panel.

📖 Key Terms in This Chapter

catalogue • product • category • SKU • model • normalisation • junction table • many-to-many • localisation • materialised path • denormalisation • index • full table scan • filesort • correlated subquery • pagination

6.1 One Product Is Not One Row

Ask a student to design a products table and you get something like this:

```
CREATE TABLE product (
  product_id INT PRIMARY KEY,
  name       VARCHAR(255),
  description TEXT,
  price      DECIMAL(10,2),
  image      VARCHAR(255),
  category   VARCHAR(100)
);
```

It is a reasonable first attempt and every part of it is wrong in a way that matters. Your shop has **eighteen** tables whose names begin `oc_product`, and the reasons for each are worth knowing, because the same reasons will apply to whatever you build after this course.

```
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'opencart'
  AND table_name LIKE 'oc_product%'
ORDER BY table_name;
```

💡 Four questions that split a table

Every one of those eighteen tables exists because of one of four questions. Learn the questions and the table list stops being arbitrary.

1. **“Does this fact depend on the language?”** A price does not; a name does. So `price` lives in `oc_product` and `name` lives in `oc_product_description`, keyed on *both* the product and the language.
2. **“Can there be more than one?”** A product has one price and many photographs, so photographs are rows in `oc_product_image` rather than columns.
3. **“Does this relate two things that both exist already?”** A product belongs to several categories and a category holds several products. Neither can hold the other, so a junction table — `oc_product_to_category` — holds the relationship.
4. **“Is this about the product, or about something that happened to it?”** `oc_product_viewed` and `oc_product_report` record events, not properties, and grow without bound.

6.1.1 The two tables you will use constantly

```
DESCRIBE oc_product;           -- the language-independent facts
DESCRIBE oc_product_description; -- the words, per language
```

`oc_product` holds what is true regardless of who is reading: `price`, `quantity`, `weight`, `status`, `date_available`, `sort_order`. Its primary key is `product_id`.

`oc_product_description` holds `name`, `description`, `meta_title` and the rest — and its primary key is the *pair* (`product_id`, `language_id`). One product, one row per language.

🔗 Why the split is not over-engineering

It looks like extra work for a shop that will only ever be in English. It is not, and the reason is worth more than the example.

Suppose `name` and `price` shared a row, and the shop later added Urdu. You now need a second row for the Urdu name — which duplicates the price. Change the price and you must remember to change it in both rows, and the day somebody forgets, the shop charges two different prices depending on the language the customer is reading. That is not a bug that gets noticed quickly; it is a bug that gets noticed by an accountant, months later.

The split makes that mistake *impossible to express*. There is exactly one place the price lives, so there is exactly one price. This is what normalisation is for, and it is the clearest small example of it you will meet.

6.2 The Screen, and What It Writes

Create a product: **Catalog > Products > Add New**. The form has tabs — **General**, **Data**, **Links**, **SEO** — and the tabs are not a presentational choice. They are very nearly the table list.

Tab	What you type	Where it lands
General	Product name, description, meta title and tags	One row in <code>oc_product_description</code> , for the language you were editing
Data	Model, price, quantity, weight, dimensions, date available, status	The single row in <code>oc_product</code>

Tab	What you type	Where it lands
Links	Manufacturer, categories, stores, related products	Rows in <code>oc_product_to_category</code> , <code>oc_product_to_store</code> and <code>oc_product_related</code> — one per tick
Image	Main image and additional images	<code>image</code> on the product row; the extras become rows in <code>oc_product_image</code>
SEO	The URL keyword	<code>oc_seo_url</code> — not a product table at all, which Chapter ?? explains

⚠ Do this once, deliberately, and you will never be confused again

Create one product. Then, in the database console at `http://localhost:8091/`, run:

```
SELECT MAX(product_id) FROM oc_product; -- yours, the newest
```

and look at that id in `oc_product`, then in `oc_product_description`, then in `oc_product_to_category`. Three tables, one product, and you typed it all on one screen.

Every chapter after this one assumes you can do this. When something in the shop is not what you expected, the question is always “what did it actually write?”, and the admin panel is the wrong place to look for the answer.

6.3 Categories, and the Table That Is Not Normalised

Categories form a tree: **Components** > **Monitors** sits under **Components**, which sits at the top. The obvious storage is a `parent_id` on each category, and that is indeed what `oc_category` has.

It is not enough, and the reason is the one interesting thing about category storage.

⚠ The question a parent pointer cannot answer cheaply

A customer opens **Components**. The page must show every product in *Components or anything beneath it*, however deep.

With only `parent_id`, answering that means: find the children of Components; find their children; find theirs; stop when there are no more. That is a loop with a query inside it, and the number of queries depends on how deep the tree happens to be — so the page gets slower as the shop’s categories get better organised, which is precisely the wrong incentive.

So OpenCart keeps a second table that stores the answer directly.

```
DESCRIBE oc_category_path;
-- category_id, path_id, level

SELECT * FROM oc_category_path WHERE category_id = 27 ORDER BY level;
-- category_id path_id level
--      27      20      0      <- Components, the ancestor
--      27      27      1      <- Monitors, itself
```

📖 Materialised Path

A **materialised path** stores, for every node, the complete list of its ancestors — one row per ancestor, with the depth. The tree is still in `parent_id`; this table is a *precomputed answer* to “who is above me?”, and therefore also to “who is below me?”

It is a deliberate **denormalisation**: the same fact is now stored twice, in `oc_category.parent_id` and in `oc_category_path`. That buys a single-query answer to the

descendants question, and costs the obligation to keep the two in step — so moving one category in the admin panel rewrites every path row beneath it.

This is the trade-off in miniature: **denormalisation buys read speed with write complexity and the risk of disagreement.** Make it deliberately, and only where the reads vastly outnumber the writes — which for a category tree they do, by perhaps a million to one.

6.4 What the Category Page Actually Asks

Here is where this chapter stops being about design and starts being about evidence. The shop's own source is in your container, and you can read it:

```
docker compose exec shop \
  cat /var/www/html/catalog/model/catalog/product.php
```

The listing query is built up in `getProducts()`, and reduced to its skeleton it looks like this:

```
SELECT DISTINCT *, pd.name, p.image,
  (SELECT ... FROM oc_product_discount pd2
   WHERE pd2.product_id = p.product_id ... LIMIT 1) AS discount,
  (SELECT ... FROM oc_product_discount ps
   WHERE ps.product_id = p.product_id ... LIMIT 1) AS special,
  (SELECT pr.points FROM oc_product_reward pr
   WHERE pr.product_id = p.product_id ...) AS reward,
  (SELECT COUNT(*) FROM oc_review r
   WHERE r.product_id = p.product_id ...) AS reviews
FROM oc_category_to_store c2s
LEFT JOIN oc_category_path cp ON (cp.category_id = c2s.category_id)
LEFT JOIN oc_product_to_category p2c ON (p2c.category_id = cp.category_id)
INNER JOIN oc_product_to_store p2s ON (p2s.product_id = p2c.product_id)
LEFT JOIN oc_product p ON (p.product_id = p2s.product_id
                        AND p.status = 1
                        AND p.date_available <= NOW())
LEFT JOIN oc_product_description pd ON (p.product_id = pd.product_id)
WHERE ...
ORDER BY p.sort_order
LIMIT 0, 20;
```

Four things in that query are worth naming, because each is a decision somebody made and each has a consequence you can measure.

1. **SELECT DISTINCT *** — every column of every joined table, then deduplicated. The **DISTINCT** is there because the path join can produce a product twice; it forces a temporary table.
2. **Four correlated subqueries.** Discount, special, reward and review count are each evaluated *per row returned*. With **LIMIT 20** that is eighty extra queries' worth of work per page, and it is the SQL form of the N+1 problem.
3. **ORDER BY p.sort_order** is the default sort, and **oc_product** has **no index on sort_order** — or on **price**, **status** or **date_available**. It has a primary key and nothing else.
4. **The whole join runs twice.** `getTotalProducts()` repeats it with **COUNT(DISTINCT p.product_id)**, because the page needs to know how many pages there are. Every category page you have ever loaded ran that join twice.

🔗 Reading EXPLAIN on the real query

Put **EXPLAIN** in front of it and the database says what it intends:

```
p:ALL [Using where; Using temporary; Using filesort]
```

Three findings in one line. **ALL** is a full table scan of `oc_product` — every row read, at any catalogue size. **Using temporary** is the `DISTINCT` building a temporary table. **Using filesort** is the `ORDER BY` sorting the result afterwards, because no index offers it in that order.

Now ask for the same page sorted by name, which is `ORDER BY LCASE(pd.name):`

`pd:index [Using where; Using index; Using temporary; Using filesort]`

Still a filesort — and `oc_product_description` *does* have an index on `name`. Chapter 6's key lesson is in that contradiction, and the next section measures it.

💡 A function around a column hides the column

`ORDER BY LCASE(pd.name)` cannot use an index on `name`.

An index on `name` stores the *names*, in order. The query does not ask for names in order — it asks for `LCASE(name)` in order, and the database has no index on that. It cannot assume the two orderings agree, because for some collations they do not. So it reads every row, computes `LCASE` on each, and sorts the results.

The rule, which is worth more than this example: the moment you wrap a column in a function in `WHERE` or `ORDER BY`, any index on that column stops being usable. `WHERE YEAR(date_added) = 2026` and `WHERE UPPER(email) = ?` are the same mistake, and they are extremely common.

The fixes are to store the value you actually search by, to use a case-insensitive collation so `LCASE` is unnecessary, or — in modern MariaDB and MySQL — to build an index on the expression itself.

6.5 What It Costs, and What an Index Buys

The argument so far is that this query ought to be slow and that an index ought to help. Both halves are worth checking, and only one of them survives.

📊 Measured: The Category Page Against Catalogue Size

Produced by `code/m06_catalogue.py` on the machine in Appendix A. Products are seeded into one category; the page is requested seven times at each size and the median taken; then the same page is requested once more with MariaDB's query log recording *every* statement it ran and how long each took.

The whole run was done **twice**, because one of its results was surprising enough that a single observation would not have been worth reporting. Both runs are shown.

Products	Page	Queries	DB time	DB as % of page
100	112 / 111 ms	144	60 / 35 ms	54% / 32%
1,000	203 / 101 ms	144	60 / 60 ms	30% / 60%
10,000	171 / 302 ms	144	114 / 128 ms	67% / 43%
50,000	541 / 476 ms	146 / 144	402 / 477 ms	74% / 100%

One page, one hundred and forty-four queries — and that is the most stable number in the whole experiment. It is the same at a hundred products as at fifty thousand, in both runs. Twenty products are displayed, each needing its images, its discount, its special price and its review count, and around forty more queries fetch settings, languages, currencies, layout modules and the category tree. The $N + 1$ problem here is per *displayed* product, which is why growing the catalogue does not grow it.

The catalogue's size does cost, and it lands on the database. Five hundred times the products cost roughly $4\times$ the page time and $8\text{--}13\times$ the database time, and the database's share rises from about a third to between 74% and 100%. At fifty thousand products this page is database-bound; at a hundred it is not.

Now the surprise. The indexes help in the middle and hurt at the top.

Products	DB before	DB after	Change (run 1 / run 2)
100	60 / 35 ms	43 / 42 ms	−28% / +19% — noise
1,000	60 / 60 ms	41 / 48 ms	−31% / −20%
10,000	114 / 128 ms	107 / 106 ms	−6% / −18%
50,000	402 / 477 ms	616 / 574 ms	+53% / +20%

At a thousand and ten thousand products the indexes make the database faster, in both runs. At fifty thousand they make it **slower**, in both runs. The direction reproduces even though the magnitudes do not, which is what makes it worth believing.

Why an index can lose. EXPLAIN shows what changed: `p:ALL [... Using filesort]` became `p:index [...]`. Given an index on `sort_order`, the optimiser walks the index in order and avoids the sort — but it must then fetch each row by primary key, which is random access. Without the index it reads the table straight through, which is sequential, and sorts afterwards. The `DISTINCT` means it has to touch nearly every row either way, so the question is only whether those reads are sequential or scattered. At fifty thousand rows, **scattered reads plus no sort lose to sequential reads plus a sort**.

The chapter's conclusion, which is not the one the chapter expected. “Add an index” is not a diagnosis, and removing a filesort is not automatically a win. The index is right for a mid-sized catalogue and wrong for a large one, and nothing in EXPLAIN told us that — only the clock did. Meanwhile the page still runs 144 queries at every size, which no index touches at all. That is a structural problem, and Chapter ?? meets it with the only tool that works at that level: caching.

⚠ Three earlier versions of this measurement were wrong

All three are recorded because all three mistakes are ones you will make. The third is the worst kind.

Third, and the one that should frighten you: the products were not there. The seeder wrote `oc_product_to_store` with `store_id = 1`. The default store is `0`. Every storefront query filters on `p2s.store_id = config_store_id`, so fifty thousand products sat in the database, appeared in the admin panel, and were invisible to every customer — and to the measurement, which was timing a category page showing the nineteen demonstration products.

Nothing failed. No error, no warning, no empty page: the page rendered perfectly, returned 200, and meant something else. The numbers it produced were real numbers about a different

question, and they were only caught because a *later* chapter tried to open one of the seeded products and got a 404.

That is the characteristic shape of a measurement bug. It does not crash; it quietly answers a question you did not ask. The defence is to check that your setup did what you think before you trust what it tells you — here, one `curl` for a seeded product's page would have caught it in week one.

First: the harness timed itself. It ran the listing query twenty-five times through `docker compose exec` and divided the wall clock by twenty-five. It reported **707 ms of query time inside a 277 ms page** — impossible, and the impossibility was the only reason it was questioned. What it was really measuring was the cost of spawning a process and opening a connection, which dwarfs the query. Timing a trivial statement first and subtracting it fixed the small sizes.

Second, and worse: it was timing the wrong query. The harness used a hand-written query that *resembled* the one OpenCart runs. At 50,000 products it still reported more database time than the whole page took, because the hand-copied version lacked the real one's restrictions and did far more work. Two queries that look alike are not alike.

The fix was to stop guessing. MariaDB will record every statement it executes, with its duration, if you set `long_query_time=0` and turn the slow query log on. That is what produced the table above — the real queries, timed by the server, including the 146 of them. Two rules, both expensive to learn any other way. **If a measurement gives you a number that cannot be true, believe the impossibility. And never hand-copy the query you mean to measure** — ask the server what it ran.

! Common Pitfalls

- **Putting the name in `oc_product`.** It is the one column that depends on the language, and the one students most want to move.
- **Editing `oc_category_path` by hand.** It is derived. Move categories in the admin panel and let it rewrite the paths, or the tree and the paths disagree and the storefront shows products in categories they are not in.
- **Wrapping a column in a function and expecting the index to work.** `LCASE(name)`, `YEAR(date_added)`, `UPPER(email)` — all the same mistake.
- **Adding indexes because a page is slow.** Measure first. This chapter added three, removed a full scan and a filesort, and gained nothing.
- **Forgetting that the join runs twice.** Optimising the listing query and ignoring the `COUNT` halves your improvement at best.
- **Testing a catalogue with nineteen products.** Every interesting behaviour in this chapter is invisible below about a thousand.

🔧 Try It Yourself: Fifty thousand products, and where the time goes

Your shop has nineteen products. By the end of this laboratory it will have had fifty thousand, and you will know what that costs.

```
# --- 1. Look at what one product is, before changing anything -----
# Create a product in the admin panel: Catalog > Products > Add New.
# Fill in General (name), Data (model, price) and Links (a category).
# Then find it three times, in three tables.
cd code
docker compose exec db mariadb -ushop -pshoppw opencart

# --- 2. Count the tables one product touches -----
```

```

# 18 of them begin oc_product. Read the list and, for each, say which
# of the four questions in this chapter put it there.

# --- 3. Fill the catalogue -----
# Seeded from a fixed seed, so your classmate's shop matches yours.
python seed/catalogue.py 50000

# --- 4. Load the category page and feel it -----
# http://localhost:8090/index.php?route=product/category&path=20
# Then sort by name, and by price. One of those is slower; predict
# which before you try it.

# --- 5. Ask the database what it is doing -----
# EXPLAIN the listing query. Find ALL, Using temporary, Using filesort.
# Then EXPLAIN it with ORDER BY LCASE(pd.name) and explain why the
# existing index on name does not save you.

# --- 6. Run the measurement -----
# Four catalogue sizes, with and without the missing indexes. Watch
# the query COUNT as the catalogue grows -- it is the finding.
python m06_catalogue.py

# --- 7. Disagree with the book -----
# The book found 146 queries per page, and that the indexes halved
# the database time without reliably moving the page time. Does
# yours? Then answer the question the book leaves open: of those 146
# queries, how many are for the 20 products shown, and how many would
# still run on an EMPTY category? Find out by requesting a category
# with no products in it, with the query log on.

# --- 8. Put your shop back -----
python seed/catalogue.py --restore

```

What to notice. Step 7 is the real exercise. The measurement in this book is a claim about one machine on one afternoon, and your job is not to reproduce the number but to find out whether the *shape* holds on yours — and, if it does, to say where the time is actually going. A student who can answer that can optimise a shop; one who can only add indexes cannot.

🔍 Checkpoint — Test Yourself

1. A shop adds Urdu. Which of these move to `oc_product_description` and which stay in `oc_product`, and why: price, product name, weight, meta description, stock quantity?
2. `oc_category_path` stores facts already derivable from `oc_category.parent_id`. Name what this buys, what it costs, and the one operation that makes the cost visible.
3. A colleague reports that a category page is slow, runs `EXPLAIN`, sees `Using filesort`, adds an index, and the page is no faster — although the database's own time halved. Give two distinct explanations for why the page did not improve, and say what you would measure to tell them apart.

📋 Chapter Summary

- One product occupies eighteen tables. Four questions decide which: does the fact depend on language, can there be more than one, does it relate two existing things, and is it a property or an event?
- `oc_product` holds the language-independent facts; `oc_product_description` holds the words, keyed on (`product_id`, `language_id`). The split makes a price that differs by language impossible to express.

- The admin form's tabs are very nearly the table list. Create one product and find it in three tables; every later chapter assumes you can.
- A parent pointer cannot answer “everything beneath this category” in one query, so `oc_category_path` stores the answer — a deliberate denormalisation buying read speed with write complexity.
- The category listing query does `SELECT DISTINCT *` with four correlated subqueries, sorts on an unindexed column, and the whole join runs twice because the page needs a count.
- `ORDER BY LCASE(name)` cannot use the index on `name`. A function around a column hides the column from the optimiser.
- Measured: **one category page runs 146 queries**, and that number barely moves between a hundred products and fifty thousand. The $N + 1$ is per displayed product, not per catalogue product.
- Five hundred times the catalogue cost about $2.5\times$ the page time, because `LIMIT 20` bounds the work.
- The three missing indexes cut database time by 45–62% at every size, and the page time did not reliably follow — the gain is real and smaller than the run-to-run noise. Measure the database to see it.
- An index makes one query faster; it cannot make 145 others disappear. The page's real problem is structural, and Chapter ?? meets it with caching.

Review Questions

1. (MCQ) The primary key of `oc_product_description` is: (a) `product_id`; (b) (`product_id`, `language_id`); (c) `name`; (d) an auto-increment id.
2. (MCQ) Using `filesort` in `EXPLAIN` means: (a) the query wrote a temporary file to disk; (b) the result had to be sorted after fetching, because no index gave that order; (c) the table has no primary key; (d) the query failed.
3. (Short) Give the four questions that decide which table a product fact belongs in, with one example each.
4. (Short) Explain why `WHERE YEAR(date_added) = 2026` cannot use an index on `date_added`, and rewrite it so it can.
5. (Short) What is a materialised path, and what has to happen when a category is moved in the admin panel?
6. (Applied) A shop wants products to carry a “minimum order quantity” that differs per customer group. Say which existing table this belongs in or what new table is needed, give the columns and the primary key, and justify it against the four questions.
7. (Applied) The measurement in this chapter found that removing a full table scan and a `filesort` did not speed the page up. Design an experiment that would locate where the time actually goes, using only what is in your container. State what you would vary, what you would hold fixed, what you would record, and what result would point at each of the four layers in Figure 5.1.

Product Variations, Options and Uploads

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Standing the Shop Up

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** an option from a variant, and **say** which of the two a given requirement needs.
- **Explain** why options do not store the combinations they imply, and **state** what that makes impossible.
- **Configure** option sets on a product and **predict** the number of rows each one creates.
- **Describe** what a shop must check before accepting an uploaded file, and **identify** which of those checks a client can defeat.
- **Test** your own shop's upload handling and **report** what it accepts.

🔗 Before You Start

Chapter 6: the eighteen product tables, and the habit of looking at what a screen wrote. Your shop should still have its demonstration catalogue; `seed/catalogue.py -restore` puts it back if not.

📖 Key Terms in This Chapter

option • option value • variant • SKU • modifier • price prefix • combinatorial explosion • stock keeping • required option • file upload • allow-list • MIME type • content sniffing • stored cross-site scripting

7.1 Two Ways to Say “Comes in Three Colours”

A shirt in three colours and four sizes. Twelve things a customer might buy. How many rows is that?

The honest answer is that it depends on a decision nobody will prompt you to make, and that the two answers have different consequences for the rest of the shop's life.

📖 Option and Variant

An **option** is a choice attached to one product, whose values *modify* it. The shirt is one product; “Large” adds Rs. 200 and “Red” adds nothing. The twelve combinations are **not stored anywhere** — they are computed when a customer picks.

A **variant** is a product in its own right, linked to a master. “Red Large Shirt” is a row in `oc_product`, with `master_id` pointing at the parent, its own model number, its own stock and its own price. Twelve combinations are twelve rows.

OpenCart 4 supports both — `oc_product` carries `master_id`, `variant` and `override` columns alongside the older option tables. The admin panel shows options first, so options are what most shops end up with, including shops that needed variants.

7.1.1 What options actually store

Two tables do the work, and a third names the choices globally.

Table	What a row means
oc_option	A choice that exists in the shop at all — “Size”, “Colour” — with its type: <code>select</code> , <code>radio</code> , <code>checkbox</code> , <code>text</code> , <code>textarea</code> , <code>date</code> , <code>time</code> , <code>datetime</code> , <code>file</code>
oc_option_value	One possible value of it: “Large”. Shared across every product that uses the option
oc_product_option	<i>This</i> product offers <i>that</i> option, and whether it is required
oc_product_option_value	This product offers that value, and what choosing it does: <code>price</code> with a <code>price_prefix</code> of + or -, plus <code>quantity</code> , <code>weight</code> and <code>points</code> adjustments

⚠ What options make impossible

Look at where `quantity` lives: on `oc_product_option_value`, which is *per option value*, not per combination.

So the shop can know it has 40 Large shirts and 30 Red shirts. It **cannot** know how many Red Large shirts it has, because there is no row for that — and no amount of configuration will add one. Three consequences follow, and all three are discovered late:

- **No stock per combination.** You will oversell the popular combination and sit on the unpopular one.
- **No SKU or barcode per combination**, so the shop cannot speak to a warehouse system that works in SKUs — which is all of them.
- **No price history or reporting per combination.** “How many Red Large did we sell?” has no answer.

If any of those three matter, you need variants, and you need to know that *before* the catalogue is built. Migrating two thousand option-based products to variants is not a setting; it is a project.

💡 Which one does this requirement need?

A test that settles most cases in one question: **does the shop need to count, price or identify the combination on its own?**

Options are right when the choice does not change what you hold in stock: an engraving message, a delivery date, a gift-wrap tick, a cable length that you cut to order. Also when the combination genuinely shares stock — which is rarer than people assume.

Variants are right whenever the combination is a thing in a box on a shelf. Clothing, shoes, anything with a barcode. If a warehouse person could pick up exactly one combination and scan it, it is a variant.

The common mistake is to choose options because the admin panel offers them first and they are quicker to set up, and to discover in month nine — during the first real stock-take — that the shop has never known what it holds.

7.2 What Each Choice Costs

Options are cheap to store and variants are not. That much is obvious. What is worth measuring is *how fast* the difference grows, because it decides whether a catalogue is manageable.

Measured: Options Against Variants

Produced by `code/m07_variations.py` on the machine in Appendix A. Option sets are attached to one product and the product page is requested seven times, median taken; the variant column is the number of `oc_product` rows the same catalogue would need.

Options	Combin.	Value rows	Page	KB	As variants
3	3	3	87 ms	30.0	4 rows
3×4	12	7	104 ms	30.7	13 rows
3×4×5	60	12	101 ms	31.3	61 rows
6×8×5	240	19	103 ms	31.3	241 rows

Options grow by addition; variants grow by multiplication. Three option sets of 6, 8 and 5 values are **19 rows** — $6 + 8 + 5$ — and express 240 combinations. The same catalogue as variants is **241 product rows**, each with its own description row, its own store row and its own category rows. Add a sixth colour and options gain one row while variants gain forty.

And the product page does not care. From 3 combinations to 240, the page stayed at about 100 ms and grew by **1.3 kilobytes** — because the page renders the *choices*, not the combinations. Nineteen values in three dropdowns, whatever the 240 products those values imply.

So the cost of variants is not page speed; it is catalogue management. Two hundred and forty-one rows that must each be priced, stocked, described, photographed and kept in step. That is the real bill, it is paid by a person rather than a server, and it is the reason shops that should have chosen variants did not.

7.3 Uploads, and What Your Shop Will Accept

One of the nine option types is `file`. Switch it on and a customer can send you a file — a logo to print, a document to quote against, a photograph to frame.

You have now accepted arbitrary bytes from a stranger, and everything that follows is about not regretting it.

What OpenCart checks, in order

From `catalog/controller/tool/upload.php` in your own container. Read it; it is forty lines.

1. **The filename is sanitised** — everything outside `[a-zA-Z0-9_.``+]` is stripped, then `basename()` removes any path. This defeats `../etc/passwd` and it is solid.
2. **The length is checked**, 3 to 64 characters.
3. **The extension is checked against an allow-list** (`config_file_ext_allowed`). An allow-list is the right shape: anything not named is refused.
4. **The MIME type is checked against a second allow-list** (`config_file_mime_allowed`).

5. **The stored name is randomised** — the file is saved as the name plus a 32-character token, so it cannot be guessed, and the real name is kept in the database.

⚠ Check 4 is worth almost nothing, and it is not a bug in OpenCart

The MIME type it compares is `$_FILES['file']['type']` — and that value is **sent by the browser**. It is a claim made by the client about its own file. Anyone using anything other than a browser sets it to whatever passes.

The default allow-list makes this worse by including `application/octet-stream`, which means “unidentified binary” — so the honest answer for almost any file is already on the list.

This is not a criticism of OpenCart so much as a lesson about where a check must run. A check performed on data the attacker controls is not a check. The extension allow-list is real because the server derives it from the name it has already sanitised. The MIME check is advisory, and the only server-side equivalent is to inspect the file’s actual contents — `finfo_file()` in PHP, which reads the magic bytes.

⚠ Two defaults on your shop, which you can verify in five minutes

Both of these are true of the stock install in Chapter 5, and both are measured in Chapter ??.

The allow-list includes svg. An SVG is not a picture; it is an XML document, and it may contain `<script>`. A browser asked to display one from your origin will run it. The default list also carries `eps` and `ps`, which are PostScript — a programming language — and `msi`, `cab` and `rar`.

Uploads are served over HTTP. They land in `system/storage/upload/`, which is *inside* the web root. Requesting a file there returns it: a text file comes back as `text/plain` and an SVG as `image/svg+xml`. The directory cannot be listed — there is an empty `index.html` — and names carry a 32-character token, so this is not trivially exploitable. But the protection is secrecy of the name, not inaccessibility of the directory, and Apache is configured with `AllowOverride None`, so the `.htaccess` OpenCart ships cannot change it.

What to do about it is Chapter ??’s subject. The short version: remove `svg` from the list unless you need it, serve uploads through a script that sets `Content-Disposition: attachment` rather than from the filesystem, and keep the storage directory outside the web root.

💡 Two size limits, and the smaller one wins

`config_file_max_size` is a setting in the admin panel. PHP has its own, `upload_max_filesize`, in `code/shop/php.ini`, and this course sets it to 8 MB.

If the admin setting is larger, it has no effect: PHP refuses the upload before any OpenCart code runs, and the error the customer sees is unhelpful. Whenever a limit appears not to work, look for the second limit — there is nearly always a second limit, in the web server, the runtime or the proxy, and the smallest one is the one in force.

❗ Common Pitfalls

- **Choosing options when the requirement needs variants.** The symptom arrives at the first stock-take, and the fix is a migration.
- **Believing option quantity tracks combinations.** It tracks one value. Red Large and Red Small share Red’s stock.
- **Trusting the uploaded MIME type.** The client supplies it.
- **Allowing svg without thinking about it.** It is script that renders as a picture.

- **Setting a size limit in one place only.** The smaller of the admin setting and `upload_max_filesize` wins.
- **Making every option required.** The customer cannot buy without choosing, and a required text option with no sensible default costs sales.
- **Forgetting that option values are global.** Editing “Large” edits it for every product that uses it.

Try It Yourself: Options, variants, and what your shop will swallow

```
# --- 1. Look at a product that already has every option type -----
# Product 42 in the demo catalogue carries all nine. Open it in the
# admin panel: Catalog > Products > edit > Option.
cd code
docker compose exec db mariadb -ushop -pshoppw opencart

# --- 2. Count what one option set costs -----
# Add a Size option with three values, through the admin panel. Then
# count the rows it created. Predict the number before you look.
#   SELECT COUNT(*) FROM oc_product_option      WHERE product_id = 42;
#   SELECT COUNT(*) FROM oc_product_option_value WHERE product_id = 42;

# --- 3. Find the row that does not exist -----
# Set stock on two option values. Then try to write a query that
# answers "how many Red Large do we hold?". You cannot. Write down,
# in one sentence, why -- that sentence is the whole of section 7.1.

# --- 4. Measure options against variants -----
python m07_variations.py

# --- 5. Read the upload controller -----
# Forty lines. Find the five checks, and find the one the client
# controls.
docker compose exec shop \
  cat /var/www/html/catalog/controller/tool/upload.php

# --- 6. Find out what your shop allows -----
docker compose exec db mariadb -ushop -pshoppw opencart -e \
  "SELECT \'key\',\'value\' FROM oc_setting \
  WHERE \'key\' LIKE \'config_file%\';"

# --- 7. Prove the upload directory is served -----
# Put a file there yourself and fetch it. This is YOUR container; see
# the scope statement in Appendix A before you try this anywhere else.
docker compose exec shop sh -c \
  'echo hello > /var/www/html/system/storage/upload/probe.txt'
curl -i http://localhost:8090/system/storage/upload/probe.txt
docker compose exec shop rm /var/www/html/system/storage/upload/probe.txt

# --- 8. Decide, and write it down -----
# For the shop you described in Chapter 4: options or variants? One
# paragraph, with the stock question answered explicitly. Chapter 23
# marks you on whether the catalogue matches this decision.
```

What to notice. Step 3 is the one that teaches. Everybody believes option stock tracks combinations until they try to write the query, and the minute spent failing to write it is worth more than this chapter’s prose.

? Checkpoint — Test Yourself

1. A shop sells mugs with a customer-supplied photograph printed on them, in two sizes. Which choices are options and which are variants, and why? Say what the shop must hold in stock.
2. A product has option sets of 4, 3 and 2 values. How many rows in `oc_product_option_value`, and how many `oc_product` rows would the same catalogue need as variants?
3. An attacker uploads a file named `logo.svg` containing `<script>`, sending `Content-Type: image/png`. Which of OpenCart's five checks does it pass, which does it fail, and what would have to be true for this to harm a visitor?

✓ Chapter Summary

- An option modifies one product; a variant is a product of its own with `master_id` pointing at a master. OpenCart 4 supports both and shows options first.
- Options store choices, not combinations. Stock, price and weight sit on the option *value*, so there is no row for “Red Large” and no query can produce one.
- That makes three things impossible: stock per combination, a SKU per combination, and reporting per combination. If any matters, use variants, and decide before the catalogue is built.
- The test: could a warehouse person pick up exactly this combination and scan it? Then it is a variant.
- Measured: option sets of 6, 8 and 5 values are 19 rows and express 240 combinations; the same catalogue as variants is 241 product rows. Options grow by addition, variants by multiplication — and the page time is unaffected either way, because the page renders choices.
- The real cost of variants is catalogue management by a person, not server time.
- A `file` option accepts arbitrary bytes from a stranger. OpenCart sanitises the name, checks the extension against an allow-list and randomises the stored name — all sound. Its MIME check is not, because the client supplies the MIME type.
- On the stock install `svg` is allowed and the upload directory is served over HTTP. The protection is an unguessable name, not an unreachable directory. Chapter ?? fixes both.

✍ Review Questions

1. (MCQ) Stock quantity for an option lives on: (a) `oc_product`; (b) `oc_product_option`; (c) `oc_product_option_value`; (d) a combination table.
2. (MCQ) Three option sets of 5, 4 and 3 values imply how many combinations, and cost how many `oc_product_option_value` rows? (a) 12 and 12; (b) 60 and 12; (c) 60 and 60; (d) 12 and 60.
3. (Short) Give the one-question test for options against variants, and apply it to a shop selling curtains cut to a customer-specified width.
4. (Short) Why is checking `$_FILES['file']['type']` not a security control, and what is the server-side equivalent?
5. (Short) An administrator raises the upload size limit in the admin panel and large files are still refused. Explain, and say where else to look.
6. (Applied) A clothing shop has 300 styles, each in 5 sizes and 4 colours. Compute the row counts for options and for variants across `oc_product` and `oc_product_description`. Then make a recommendation, naming the single requirement that decides it.

- 7. (Applied)** Design the upload handling you would write for a shop that must accept customer artwork. State every check, say which side it runs on, say what each one defends against, and say what you would do with the file after accepting it so that serving it later is safe.

Themes, Layout and the Customer's Experience

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Standing the Shop Up

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** how a theme is built from templates, and **override** one without editing the original.
- **Explain** what a layout is, and **place** a module on a chosen page without touching a template at all.
- **Measure** a page's weight and **say** which part of it is the weight.
- **Distinguish** the optimisations that are worth doing from the ones that are merely popular, using evidence.
- **Judge** a storefront change by its effect on the customer rather than by taste.

🔗 Before You Start

Chapter 6 and Chapter 7: a catalogue worth looking at. Appendix B's Twig section — this chapter edits templates. HTML and CSS from *Web Technologies*.

📖 Key Terms in This Chapter

theme • template • Twig • template override • layout • module • position • page weight • request count • render-blocking • minification • responsive design • above the fold • conversion

8.1 What a Theme Actually Is

“Change the theme” sounds like changing a colour. It is not. A theme in OpenCart is a directory of Twig templates, and each one is responsible for one piece of the page.

```
docker compose exec shop \
ls /var/www/html/catalog/view/template/product/
# category.twig product.twig search.twig special.twig ...
```

The default theme lives in `catalog/view/template/`. The important rule about it is one you should adopt before you change a single line:

⚠️ Never edit the default theme

Edit `catalog/view/template/product/category.twig` directly and three things happen. Your change is lost at the next upgrade. Nobody can tell what you changed, because there is no original to compare against. And a second developer, or you in March, will assume the file is stock and read it as documentation of how OpenCart works.

Make a **child theme** instead: a directory of your own that contains *only* the templates you changed. OpenCart looks there first and falls back to the default for everything else, so your directory *is* the list of your changes. A child theme with four files in it is a complete and honest statement that you changed four things.

8.2 Layouts, or How to Change a Page Without Editing It

A surprising amount of what people reach for a template to do can be done without touching one.

Layout, Module, Position

A **module** is a self-contained block — a category list, a banner, a carousel, “featured products”.

A **layout** says which modules appear on which *kind* of page, and where. **Design > Layouts** lists them: Home, Category, Product, Checkout, and so on.

A **position** is a slot in the page: **Column Left**, **Column Right**, **Content Top**, **Content Bottom**. A module is placed in a position of a layout, with a sort order.

So “put a banner above the products on every category page” is three clicks in **Design > Layouts > Category** and no code at all.

Ask this before editing a template

“Is this a layout change or a template change?”

If you want something *moved*, *added* or *removed* and it already exists as a module, it is a layout change — do it in the admin panel, where it is visible to whoever comes next and survives upgrades.

Edit a template only when the *markup itself* must differ: a field shown that the theme does not show, a different arrangement within a block, a tag the theme gets wrong. Then override exactly that file in your child theme and nothing else.

The commonest waste of effort in this chapter's laboratory is a student editing `category.twig` to do something **Design > Layouts** would have done.

8.3 What a Page Weighs

Here the chapter stops being about arrangement. A customer's experience of your shop is mostly its speed, and speed is mostly *weight* — how many bytes and how many separate requests the browser must complete before the page is usable.

Note that this is a different question from Chapter 6's. That chapter measured how long the *server* took to produce the HTML. This one measures everything that happens afterwards, and it is usually larger.

Measured: What the Storefront Weighs

Produced by `code/m08_pageweight.py` on the machine in Appendix A. The harness fetches a page, finds every stylesheet, script and image the HTML names, requests each one once, and totals them.

Page	Requests	HTML	CSS	JS	Images	Total
Home	25	32 KB	348 KB	223 KB	293 KB	895 KB
Category	20	46 KB	348 KB	223 KB	122 KB	738 KB
Product	18	54 KB	354 KB	243 KB	61 KB	713 KB

The framework is the page. CSS and JavaScript come to about **570 KB on every page**, and they do not vary with the content — the same bytes on the home page, a category of 50,000 products and a single product. On the product page they are **84% of the weight**. The HTML that the whole of Chapter 6 was spent optimising is 6–7%.

Images are not the problem here, although everyone optimises them first. They are 33% of the home page, 16% of a category page and 9% of a product page. The largest single image in the demonstration catalogue is 13.9 KB. You could halve every image on the category page and save 61 KB — about 8%.

The single largest file is 265 KB of unminified CSS. `bootstrap.css` ships at 271,637 bytes, and **no minified version ships with it** — there is no `bootstrap.min.css` anywhere in the tree. A minified build of the same framework is around a tenth of that. That one file is 36% of the category page.

This measurement undercounts, and you should know how. It follows only assets the *HTML* names. Fonts are requested by the CSS, so they are invisible to it: the FontAwesome directory alone holds 248 KB of `.woff2` files, of which a browser fetches whichever subsets it needs. The totals above are therefore a floor, not a figure. A tool that drove a real browser would see more; this one is honest about seeing less.

💡 The order to optimise in, which is not the popular order

From the numbers above, for this shop:

1. **Minify the CSS.** One file, 265 KB to roughly 27 KB. The largest single saving available, and it changes nothing a customer can see.
2. **Drop what you do not use.** Bootstrap and FontAwesome are general-purpose libraries; a shop uses a fraction of either. Removing the unused parts is the second-largest saving and the fiddliest.
3. **Cut the request count.** Twenty requests is twenty round trips. On a fast link this hardly matters; on a phone on a slow connection in Bahawalpur it is most of the wait.
4. **Then images** — resize to the size actually displayed, and serve modern formats.

Most advice reverses this list, because images are the thing people can see and a 2 MB photograph is a vivid story. On *this* shop images are 16% and the framework is 77%. On a shop with photographic product shots the balance moves. That is the point: the order depends on the measurement, and the measurement takes two minutes.

⚠️ Render-blocking, and why position matters as much as size

A stylesheet in the `<head>` is **render-blocking**: the browser will not paint *anything* until it has fetched and parsed it. So 348 KB of CSS is not merely 348 KB of transfer — it is 348 KB standing between the customer and the first pixel.

Scripts block too, unless marked `defer` or `async`. A `<script src>` in the head stops the parser dead while it downloads.

This is why page weight and page time are not the same measurement, and why “it is only 200 KB” can still be the reason a shop feels slow.

8.4 Judging a Change by the Customer

The hardest discipline in this chapter is not technical. It is resisting the urge to change the storefront because you would prefer it differently.

⚡ Three changes, and how you would know if they helped

“Make the Add to Cart button green.” You cannot know without an experiment: show half your visitors one colour and half the other, and compare the proportion who buy. With thirty visitors a day you will wait months for an answer, which is itself the answer — a shop this size should spend its effort elsewhere.

“Show the delivery charge on the product page.” This you can reason about. Chapter 1’s laboratory found that shops which hide the delivery charge until checkout lose customers there; it is the most-cited reason for abandonment. No experiment needed.

“Minify the CSS.” Measurable directly, in bytes, with no customers involved at all. 265 KB to 27 KB is a fact, and it cannot make anything worse.

The pattern. Prefer changes you can verify without an experiment you cannot afford to run. Most small shops cannot generate enough traffic to measure a colour, and should spend their effort on the things that are measurable in bytes or known from everybody else’s evidence.

! Common Pitfalls

- **Editing the default theme.** Lost at upgrade, invisible to the next person. Override in a child theme.
- **Editing a template to do a layout’s job.** If the block exists as a module, move it in **Design > Layouts**.
- **Optimising images first because they are visible.** On this shop that addresses 16% of the weight and ignores 77%.
- **Assuming a framework file is minified.** `bootstrap.css` here is 265 KB and there is no minified copy in the tree.
- **Measuring page weight by counting every `<img>` tag.** The same thumbnail on twenty cards is one request; counting it twenty times overstates the weight several-fold.
- **Forgetting fonts.** They are requested by CSS, so they are invisible to any harness that reads only the HTML — including this chapter’s.
- **Redesigning on taste with no way to tell whether it helped.**

🔧 Try It Yourself: Weigh your shop, then make it lighter

```
# --- 1. Weigh three pages -----
cd code
python m08_pageweight.py --compare

# --- 2. Find the biggest single file -----
# Predict it before you look. Most people guess an image.
docker compose exec shop sh -c \
  'ls -laS /var/www/html/catalog/view/stylesheets/ | head -5'

# --- 3. Make a child theme -----
# Admin: Design > Theme Editor. Override ONE template -- the category
# product card -- and add the model number under each product name.
# Your override directory is now the list of your changes.

# --- 4. Do a layout change with no code -----
# Admin: Design > Layouts > Category. Add a module to Content Top.
# Reload a category page. No template was edited.
```

```
# --- 5. Weigh it again -----
# Did your two changes move the number? By how much, and which one?
python m08_pageweight.py

# --- 6. Make a real saving -----
# Minify bootstrap.css and serve the minified copy instead. Any
# minifier will do. Record the before and after byte counts.

# --- 7. Weigh it a third time, and account for the difference -----
# The saving should be most of 240 KB. If it is not, find out what
# else changed -- do not just record the number.
python m08_pageweight.py

# --- 8. Say what you would do next, and why -----
# One paragraph, ordered by measured saving rather than by what is
# easiest to see. Chapter 20 returns to this with caching.
```

What to notice. Step 2 against step 1: almost everyone predicts an image and the answer is a stylesheet. That gap between where people look for weight and where weight is, is the whole chapter.

🔍 Checkpoint — Test Yourself

1. A product page is 713 KB, of which 597 KB is CSS and JavaScript. A colleague proposes compressing all the product photographs. What is the best case for that change, and what would you propose instead?
2. Distinguish a layout change from a template change, and give one example of each for the request “show a trust badge under the Add to Cart button on every product page”.
3. This chapter’s harness reports 738 KB for a category page, and a browser’s developer tools report more. Give two reasons the harness is lower, and say which of them it could fix.

📋 Chapter Summary

- A theme is a directory of Twig templates. Never edit the default: override only the files you change in a child theme, so that the directory is the list of your changes and survives upgrades.
- A layout places modules in positions on a kind of page. Moving, adding or removing an existing block is a layout change made in the admin panel, not a template edit.
- Measured: the storefront weighs 713–895 KB per page across 18–25 requests. CSS and JavaScript are about 570 KB of that *on every page* and do not vary with content — 84% of the product page.
- Images are 9–33% depending on the page, although they are what people optimise first.
- The largest single file is `bootstrap.css` at 265 KB, unminified, with no minified copy anywhere in the tree.
- The harness undercounts: it follows only assets named in the HTML, so fonts requested by CSS (248 KB in one directory) are invisible to it. Know what your instrument cannot see.
- Weight and time are not the same: a stylesheet in the head is render-blocking and stands between the customer and the first pixel.
- Prefer changes you can verify without an experiment you cannot afford to run. A small shop cannot measure a button colour and can measure 240 KB.

 Review Questions

1. **(MCQ)** On the measured product page, CSS and JavaScript are what share of the weight? (a) 9%; (b) about a third; (c) 84%; (d) it depends on the catalogue size.
2. **(MCQ)** “Show the manufacturer’s name under the product title” where the theme does not show it at all is: (a) a layout change; (b) a template override; (c) a module setting; (d) a database change.
3. **(Short)** Why is a child theme better than editing the default, giving two reasons that are not about upgrades.
4. **(Short)** Explain render-blocking, and why page weight and page time are different measurements.
5. **(Short)** This chapter’s harness counts each unique asset once rather than once per mention. Explain what would go wrong otherwise, with a number.
6. **(Applied)** You are given a shop whose category page is 2.1 MB: HTML 40 KB, CSS 90 KB, JS 150 KB, images 1.82 MB across 40 requests. Give your optimisation order with a predicted saving for each step, and say how this differs from the order you would use on the shop in this chapter and why.
7. **(Applied)** Design a measurement that would tell you whether showing the delivery charge on the product page increases or decreases completed orders. State what you would record, how long you would need to run it, and — honestly — whether a shop taking thirty orders a day could reach a conclusion at all.

PART III

From Basket to Order

A catalogue is a brochure until somebody can buy from it. Part III is the path from a product page to a confirmed order: where the basket lives, what the checkout is really doing, and the two things that change the total after the customer thought they knew it — shipping with tax, and discounts.

the order state machine: everything after it is a transition

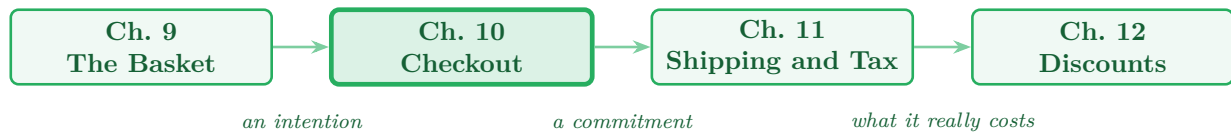


Figure P3.1: Reading path through Part III. Chapter 10 is the hinge of the whole book — an order is the first object in the course that cannot simply be deleted and redone.

The Shopping Basket, and Where State Lives

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — From Basket to Order

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why HTTP has no memory, and **name** the three places a shop can keep what it remembers about a visitor.
- **Locate** a basket in a running shop, and **state** which store holds it, with evidence rather than with belief.
- **Measure** what a session costs in rows, bytes and queries, and **distinguish** the cost of a visitor from the cost of a customer.
- **Predict** what survives a restart, a redeployment and a volume deletion, and **verify** each prediction.
- **Judge** a session configuration — engine, expiry, cookie flags — against what the shop actually does.

🔗 Before You Start

Chapter 5: containers, volumes, and the difference between restarting a container and deleting one. Chapter 6: the tables behind the screens, and how to read a query the server actually ran rather than one you assumed it ran. From Web Technologies: HTTP requests and responses, and what a cookie is.

🔑 Key Terms in This Chapter

state • stateless protocol • cookie • session • session identifier • session engine • server-side state • client-side state • persistent cart • abandoned basket • session expiry • garbage collection • volume • cache miss

9.1 The Problem: A Protocol With No Memory

A customer puts a laptop in the basket, browses to another category, comes back, and the laptop is still there. Nothing about HTTP makes that happen.

Every HTTP request is independent. The server receives a request, answers it, and keeps nothing. The next request from the same person arrives looking exactly like a request from a stranger. If the shop is to know that these two requests belong to the same visitor, *something carried between them* must say so, and that something can only travel in the request itself — because the request is all there is.

📖 State, Cookie, Session

State is anything the shop remembers between one request and the next: what is in the basket, who is logged in, which currency is being shown.

A **cookie** is a small named value the server asks the browser to store and send back on every subsequent request to that site. It is the only standard way to make a second request

identifiably related to a first. It lives on the customer's machine, which means the customer can read it, change it and delete it.

A **session** is state the *server* keeps, filed under a key. The browser carries only the key — the **session identifier** — in a cookie; the data stays on the server, where the customer cannot touch it.

The distinction is the whole of this chapter. A cookie is a place. A session is an arrangement that *uses* a cookie to point at a place somewhere else.

9.2 Three Places State Can Live

Given that the browser must carry something, a shop has three choices about *how much* it carries, and they trade off against each other in ways worth seeing before measuring anything.

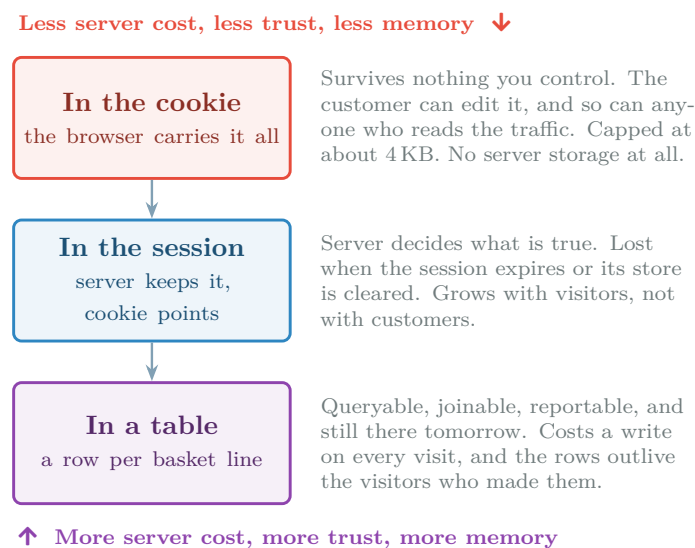


Figure 9.1: The three stores, and what each one buys. Nothing forces a shop to pick one: most use all three at once, for different things. The question worth asking of any shop is not which one it uses, but which one holds the basket.

💡 A rule of thumb, before any measurement

Put in the **cookie** only what you would not mind the customer editing — a chosen language, a dismissed banner. Put in the **session** what must be true for this visit and need not outlive it. Put in a **table** anything you will later want to count, report on, or still have tomorrow. Applying that rule to a basket gives an answer immediately: a shop wants to know how many baskets were abandoned last week, so the basket belongs in a table. That is an argument. The rest of this chapter is evidence.

9.3 Where This Shop Actually Puts It

Almost every tutorial on this subject says the cart is kept in the session. Before believing that of the shop in front of you, look.

The method is the one Chapter 6 arrived at the hard way: do not reason about what the code probably does, make the shop do it and read what changed. Add three products, and after each one measure all three stores at once.

🔍 Measured: Add three products; watch all three stores

Produced by `code/m09_basket.py` -where against the stack in Appendix A. “Cookie” is what the browser sends up on every request; “session” is the length of the stored session record; “cart” is rows in `oc_cart` and the bytes in them.

After adding	Cookie	Session	Cart rows	Cart bytes
(nothing)	49 B	18 B	0	0
2 × MacBook	49 B	18 B	1	53
1 × iPhone	49 B	18 B	2	106
3 × HTC Touch HD	49 B	18 B	3	159

The cookie does not move, and the session does not move. Both are constant across an empty basket and a three-line one. The entire session record is eighteen bytes:

```
{"currency": "USD"}
```

The basket is a table. It is rows in `oc_cart`, one per line, about 53 bytes each, keyed by the session identifier. The cookie carries a 26-character identifier and a currency, 49 bytes in all, and that is the whole of what the browser knows.

So the common claim is false here. It is worth knowing *why* it is so widely believed: in a simpler shop it is usually true, and the arrangement OpenCart uses — session for identity, table for the basket — is the one a shop adopts when it wants to report on baskets that were never checked out.

⚠️ “The cart is in the session” — check before you repeat it

This matters beyond trivia. If you believe the basket is in the session, you will also believe:

- that lengthening the session expiry lengthens the life of a basket (here it does both, but for two different reasons, in two places);
- that switching the session engine moves the basket (it does not — see the next measurement);
- that clearing sessions clears baskets (it does not; it orphans them);
- that you cannot report on abandoned baskets without extra work (you can: they are rows, and Chapter ?? reads them).

Each of those is a decision someone will make on a Friday afternoon. The evidence took one command.

9.4 What a Session Costs

The basket is in a table, but the shop still keeps sessions, and sessions are not free. Two questions: how many are created, and what does each one cost per request?

🔍 Measured: A session is written before the visitor asks for anything

From `code/m09_basket.py` -sessions. Twenty requests made as twenty first-time visitors, then twenty made by one visitor who keeps the cookie.

20 first-time visitors	20 new session rows
1 visitor, 20 page views	1 new session row

The session table on this machine held **645 rows, about 35.6 KB**, accumulated entirely by the measurements in this handout — no human has ever shopped here.

Every request writes a session, including the first. A visitor who loads one page and leaves has cost a row. A crawler that ignores cookies costs one row per URL it touches, and the rows are removed on a clock rather than on departure: `config_session_expire` is **86,400 seconds**, a full day.

This is the first number in the course that scales with *traffic* rather than with *business*. The catalogue in Chapter 6 grew because the shop sold more things. This grows because more robots visited.

🔧 Measured: Database sessions against file sessions

From `code/m09_basket.py -engine`. OpenCart ships three session engines — `db`, `file` and `redis` — selected in `system/config/catalog.php`. The storefront ships configured for `db`. Both rows below are the same home page, one engine apart.

Engine	Page	Queries	On session	Cart rows	Files
db	89 ms	117	2	1	1
file	71 ms	115	0	1	14

Exactly two queries per request are the session's — one `SELECT` to read it and one `REPLACE` to write it back. Moving to files removes both and takes about 20% off the page on this machine, where the database and the web server are two containers on one host.

The basket did not move. The `Cart rows` column is 1 under both engines. The engine decides where eighteen bytes live; it has nothing to do with the basket.

Files are not free either, they are just billed elsewhere. Each session file holds those same eighteen bytes and occupies a 4 KB filesystem block — an amplification of more than two hundred. And a file session is tied to one machine's disk, so the moment the shop runs on two web servers, a customer served by the second one is a stranger. That, not speed, is why the shipped default is `db`.

💡 Reading that 20% honestly

It was measured on one host where the database is a container a virtual network hop away. On a deployment where the database is a separate machine, removing two round trips per request would save more. On a deployment behind a load balancer, the file engine would not work at all.

A measurement tells you what happened on the machine it ran on. Appendix A names that machine for exactly this reason. Quoting 20% as a property of OpenCart would be a misuse of your own result.

9.5 Two Things the Source Does That the Screens Do Not Show

Reading `system/library/cart/cart.php` and `system/library/session.php` turns up two behaviours that no admin screen mentions and both measurements confirm.

🔧 Measured: An empty basket costs more than a full one

From `code/m09_basket.py -cache`. The same home page, with the basket holding 0, 1, 2 and 3 lines.

Basket lines	Queries	SELECT on cart	DELETE on cart
0	117	6	1
1	119	1	1
2	126	1	1
3	129	1	1

An empty basket is read six times; a full one is read once. The cache in `Cart::getProducts()` begins if `(!$this->data)`, which is true both when the basket has not been loaded yet *and* when it is empty. PHP cannot tell those apart that way, so an empty basket misses its own cache on every call — and a shop’s commonest visitor has an empty basket.

Every storefront page deletes before it reads. The DELETE column is 1 on every row, including the empty basket. The `Cart` constructor removes expired guest baskets each time it is built, so a page view is a write even when the visitor only looked.

🔧 Measured: One request in five rebuilds the session table

From `code/m09_basket.py -gc`, over two runs.

25 requests	OPTIMIZE ran 7 (28%)	555 ms in total
60 requests	OPTIMIZE ran 15 (25%)	889 ms in total

`Session::__construct` registers `gc()` as a shutdown function, so garbage collection is *considered* on every request; `session_probability` and `session_divisor` make it *run* on about one in five. When it runs it deletes expired rows and then issues `OPTIMIZE TABLE`, which on InnoDB is a full table rebuild — around 60 ms here, on a 645-row table.

Both runs are printed because neither is the answer. The configured rate is one in five; 7 of 25 and 15 of 60 are what one in five looks like in small samples, and reporting only the first would have claimed 28%.

9.6 What Survives

Chapter 5 introduced volumes with the claim that they are what makes a container’s data outlive the container. A basket is the smallest honest test of that claim, because you can put one somewhere and then try to destroy it three different ways.

🔧 Measured: Three ways to stop a shop, and what is left afterwards

From `code/m09_basket.py -restart -destroy`. A three-line basket is created, then each operation is applied in turn and the basket looked for again — in the tables, and in the page the same browser is shown.

Operation	Took	Cart rows	Session rows	Basket shown
— (before)	—	3	1	yes
restart shop	3 s	3	1	yes
down, up -d	10 s	3	1	yes
down -v, up -d	17 s	0	0	no

Destroying the containers destroyed nothing. `down` removes the containers; the named volumes stay, so the database files stay, so the basket stays. The browser's cookie still pointed at a session that still existed.

`-v` is the whole difference. It deletes the volumes. The shop reinstalled itself from scratch in 17 seconds — which is a good property of the stack and a bad surprise for anyone who typed it to free disk space.

Notice what the third row does *not* say. It does not say the shop broke. It came back, fast, looking perfectly healthy, with every customer's basket gone. Chapter ?? returns to this with backups, where the lesson is the same one Chapter 6 taught about the store identifier: a shop that answers 200 is not thereby a shop that is right.

9.7 The Settings That Govern Any of This

Three of the behaviours above are configurable, and two of the three are in the admin panel rather than in a file.

Where each decision is actually made

- **How long an abandoned basket lives.** **System > Settings > Edit > Option** — the session expiry, `config_session_expire`, 86,400 seconds by default. It governs both the session record and the DELETE in the `Cart` constructor, so shortening it discards baskets sooner.
- **Whether a basket needs an account at all.** **System > Settings > Edit > Option** — **Guest Checkout**. With it on, the basket is keyed by session alone and vanishes with the cookie. With it off, every basket belongs to a customer and follows them to another machine.
- **Where the session itself is kept.** Not in the admin panel at all, but in a file: the `session_engine` line of `system/config/catalog.php`. A setting that changes both a performance characteristic and a deployment constraint, and that can be reached only by editing a file, is worth noticing as a design choice — it is not meant to be changed by the shopkeeper.

Common Pitfalls

- **Assuming where the basket is instead of looking.** Every wrong decision in this chapter follows from that one.
- **Putting a price in the cookie.** Anything the browser carries, the customer can edit. `oc_cart` has a `price` column and the shop recomputes rather than trusting it; your own extensions must do the same.
- **Lengthening session expiry to “keep baskets”** without noticing that it also keeps every robot's session row for the same period.
- **Clearing `oc_session` to tidy up.** It does not delete baskets; it strips them of the identifier that reaches them, leaving rows nothing will ever read and nothing will ever remove.
- **Typing `docker compose down -v` to free space.** It is the one command in this chapter that cannot be undone.
- **Reading a timing from one machine as a property of the software.** The 20% above is a fact about this host.

Try It Yourself: Find the basket, then try to destroy it

```
# --- 1. Predict, in writing, before running anything -----
# Where is the basket: cookie, session, or a table? Commit to one.
```

```

# --- 2. Look -----
cd code
python m09_basket.py --where

# --- 3. Read the session record yourself -----
docker compose exec db mariadb -ushop -pshoppw opencart \
    -e "SELECT session_id, LENGTH(data), data FROM oc_session LIMIT 3;"

# --- 4. Find your own basket in the table -----
docker compose exec db mariadb -ushop -pshoppw opencart \
    -e "SELECT * FROM oc_cart;"

# --- 5. Move the session to files, and check the basket -----
python m09_basket.py --engine
# Did the basket move? Write down why not, in one sentence.

# --- 6. Count what an empty basket costs -----
python m09_basket.py --cache
# Explain the first row to someone who has not read the source.

# --- 7. Try to destroy it, in increasing order of violence -----
python m09_basket.py --restart
# Predict each row before you read it.

# --- 8. Now the one that works -----
# Only when you are ready to reinstall the shop:
python m09_basket.py --restart --destroy

# --- 9. Change the expiry and show that it did something -----
# Admin: System > Settings > Edit > Option. Set session expiry to 60
# seconds. Add a product, wait, reload. Record what you see and which
# of the two mechanisms in this chapter produced it.

```

What to notice. Step 1 against step 2 is the point of the lab. Keep your written prediction: the gap between what you expected and what the shop does is the only part of this exercise you will still remember in a year.

? Checkpoint — Test Yourself

1. A colleague says “move the sessions to Redis and the basket will be faster”. Using this chapter’s measurements, say what would and would not change, and give the number that settles it.
2. The session table holds 645 rows on a shop that has never had a customer. Explain where they came from, why they are not removed when the visitor leaves, and what you would measure before deciding to shorten the expiry.
3. `docker compose down` left a three-line basket intact and `docker compose down -v` did not. Explain the difference in terms of what each command removes, and name one other thing in your shop that the second command destroys.

✓ Chapter Summary

- HTTP keeps nothing between requests. A cookie is the only standard way to make the second request identifiably related to the first.
- A cookie is a place on the customer’s machine; a session is server-side state that *uses* a cookie to point at itself. Confusing the two confuses who can change what.
- Measured: adding three products left the cookie at 49 bytes and the session at 18 bytes, and added three rows to `oc_cart`. The basket in this shop is a table, not a session.

- Measured: 20 first-time visitors created 20 session rows; one visitor making 20 page views created one. Sessions scale with traffic, not with trade.
- Measured: the `db` engine costs exactly two queries per request; the `file` engine costs none and took about 20% off the page on this host — while moving no part of the basket, and while making a second web server impossible.
- Measured: an empty basket is read six times per page and a full one once, because the cache test cannot distinguish “empty” from “not loaded”. Every storefront page also deletes from `oc_cart` before reading it.
- Measured: about one request in five rebuilds the session table with `OPTIMIZE TABLE`, at roughly 60 ms.
- Measured: `restart` and `down/up` left the basket untouched; `down -v` destroyed it and the shop reinstalled itself in 17 seconds looking perfectly healthy.
- Where state lives decides what survives, what can be reported on, and who is able to tamper with it. It is a design decision, not a detail.

Review Questions

1. **(MCQ)** In the measured shop, adding a product to the basket changes: (a) the cookie; (b) the session record; (c) rows in a table; (d) all three.
2. **(MCQ)** Switching `session_engine` from `db` to `file` moved: (a) the basket; (b) the session record only; (c) both; (d) neither.
3. **(Short)** Explain why a session identifier must be unguessable, and what an attacker who obtained one could do in this shop.
4. **(Short)** Give two things that belong in a cookie and two that must never go in one, with your reason in each case.
5. **(Short)** An empty basket costs six cart queries and a full one costs one. Explain the cause, and propose a one-line change to the condition that would fix it.
6. **(Applied)** A shop reports 400,000 session rows and 12 orders a day. Work out what is producing the rows, state what you would measure to confirm it, and give two changes you would make — one to configuration and one to the shop’s robots policy.
7. **(Applied)** You are asked to make baskets survive for thirty days so that customers can come back to them. Say exactly what you would change, what each change costs, what it would break for a guest with no account, and how you would measure whether it increased completed orders at all.

Appendices

The Stack: Setting It Up and Keeping It

Everything in this book from Chapter 5 onward assumes a shop you can reach in a browser. This appendix gets you one, in a single sitting, and lists what to do when it refuses — which on a university laptop it sometimes will.

A.1 What You Install

One thing: a container runtime. Not a web server, not PHP, not a database. Those three run inside containers and never touch your own machine, which is why the shop is identical on every laptop in the class and why uninstalling it is one command.

- **Windows or macOS:** Docker Desktop, from docker.com. On Windows it needs WSL 2, which its installer offers to set up.
- **Linux:** the `docker.io` and `docker-compose-v2` packages from your distribution, then add yourself to the `docker` group and log out and in again.

Check it with:

```
docker --version
docker compose version
docker run --rm hello-world
```

The third is the one that matters. The first two only prove the program is installed; the third proves the *engine* is running, which is a different claim and the one that usually fails.

A.2 Starting the Shop

```
cd code
cp .env.example .env
docker compose up -d --build
python waitfor.py
```

The first run takes a few minutes and downloads about 1.5 GB; later runs take seconds. Chapter 5 measures both and explains why the difference matters.

What	Where	Sign in with
The storefront	<code>http://localhost:8090/</code>	nobody; you are a customer
The admin panel	<code>http://localhost:8090/admin/</code>	admin / admin123
The database console	<code>http://localhost:8091/</code>	server db, user shop, password shoppw, database opencart

⚠ Those credentials are deliberately bad

`admin / admin123` is exactly the kind of credential Chapter ?? goes looking for and finds. It is the default so that you can measure the shop before and after hardening it; Chapter 16 asks you to change it, and until then every student in the class can sign in to every other student's shop if they can reach it.

Which they cannot, because nothing here is published to the internet. If you ever expose this stack beyond your own machine, change the credentials in `.env` *first*.

A.3 The Four Commands Worth Memorising

Command	What it does
<code>docker compose up -d</code>	Start everything. Safe to run when it is already running
<code>docker compose down</code>	Stop everything. Your shop is kept
<code>docker compose logs -f shop</code>	Watch the shop's log. This is where PHP errors go, and they go nowhere else
<code>docker compose down -v</code>	Stop everything and delete your shop . The reset button, and the one mistake with no undo

A.4 When It Will Not Start

Five failures account for nearly all of them, and four are not really about Docker.

1. **“Cannot connect to the Docker daemon”** — the engine is not running. On Windows and macOS, start Docker Desktop and wait for its icon to stop animating; the command line is ready several seconds before the whale stops moving.
2. **“Ports are not available”** or “address already in use” — something else on your machine already holds port 8090 or 8091. This is common; the development machine for this book had both 8080 and 8081 taken, which is why the defaults are 8090 and 8091 in the first place. Edit `.env`, change `SHOP_PORT`, and bring it up again. Change only the port in `.env` — `OC_HTTP_SERVER` reads the same variable, so links keep working.
3. **The shop answers but every page is an error** — the install did not finish. Read `docker compose logs shop` and look for “SUCCESS! OpenCart successfully installed”. If it is absent, the database was not ready; `docker compose down -v` and start again.
4. **WSL 2 is not installed, or virtualisation is disabled** — Windows only. Docker Desktop's installer handles the first; the second is a BIOS setting (often called “Intel VT-x”, “AMD-V” or “SVM Mode”) that some laptops ship disabled.
5. **It worked yesterday and today it does not** — almost always disk space. `docker system df` will tell you; `docker system prune` reclaims what is not in use. It does not touch named volumes, so your shop survives — but read what it proposes before agreeing.

Read the log before changing anything

The single most useful habit in this appendix. A shop that is broken is broken for a reason, and the reason is almost always written down in `docker compose logs shop`.

Restarting, rebuilding and resetting are the three things students try first, and all three destroy the evidence. Look at the log, then act.

A.5 Backing Up, and Why You Will Wish You Had

Your shop lives in two named volumes. `down -v` deletes them, and so does an afternoon of clearing disk space without reading the prompt.

```
# Dump the database into the repository, where git can see it.
docker compose exec -T db mariadb-dump -ushop -pshoppw opencart \
  > backups/shop-$(date +%Y%m%d).sql

# Put it back.
```

```
docker compose exec -T db mariadb -ushop -pshoppw opencart \
< backups/shop-20261001.sql
```

Take one after every chapter's laboratory. It costs two seconds, and Chapter ?? measures what the alternative costs.

⚠ Where to keep your own work

Anything you write that must survive belongs in `code/`, in the repository — not inside a running container, where it is deleted at the next rebuild, and not only in the admin panel, where it is deleted by `down -v`.

This matters most in Chapter ??, where you write a module. Write it in `code/modules/` and mount it; do not edit it in place inside the container. Students who learn this in Week 14 learn it the expensive way.

A.6 The Machine Every Measurement Was Taken On

Every figure in a 📊 **Measured** box in this book was produced on this machine. Yours will differ; the shapes and ratios should not.

Item	Value
Host	Windows 11 Pro, 4 CPU cores, 7.7 GiB available to the container runtime
Container runtime	Docker 28.1.1, Linux containers
Web and application	Apache 2.4.68, PHP 8.2.34 with OPcache enabled
Database	MariaDB 11.8.9
Application	OpenCart 4.1.0.4, installed from the pinned release in <code>code/shop/Dockerfile</code>
Everything on	one machine, so no figure here includes real network latency

⚠ The scope of Chapter 16's security work

Chapter ?? asks you to attack a shop: SQL injection, cross-site scripting, missing security headers, default credentials, an exposed administrative path. Every one of those exercises is scoped, in the chapter and here, to **the container running on your own machine, reached at localhost, which you installed and which contains no real person's data**.

That is the whole of it. Running the same checks against any system you do not own and have not been authorised in writing to test is a criminal offence in Pakistan under the Prevention of Electronic Crimes Act 2016, and a sackable one nearly everywhere else. The skill is valuable precisely because it is dangerous, and the boundary is not a formality.

Chapter ?? returns to this with the statute in front of it.

Enough PHP, SQL and Twig to Read the Shop

The sibling courses give this appendix to mathematics. This one spends it on language, because that is what this course actually assumes and does not teach.

You are not asked to be a strong PHP programmer. Until Chapter ?? you will read far more code than you write, and this appendix is calibrated to that: enough to follow what the platform is doing, and enough to change it when Chapter ?? asks you to.

B.1 PHP, in Two Pages

B.1.1 The shape of a file

PHP is a language for producing a response to an HTTP request. A file runs top to bottom, every time the page is requested, and whatever it prints becomes the page.

```
<?php
declare(strict_types=1);

$name = 'Keyboard';           // string, single quotes: no interpolation
$price = 2499.00;             // float
$qty   = 3;                   // int
$total = $price * $qty;

// Double quotes interpolate; single quotes do not. This is the single
// most common source of confusion in a first week of PHP.
echo "Total for $name: $total"; // Total for Keyboard: 7497
echo 'Total for $name';        // Total for $name
```

⚠ Three things that surprise everyone

Variables start with \$, always — including inside strings and in function parameters.

== is not ===. The two-character form converts types before comparing, so '0' == false is true and 'abc' == 0 was true in older versions. **Use === unless you have a reason not to**, and in Chapter ?? you will see what the other one costs.

declare(strict_types=1) is not the default. Without it PHP quietly converts "5 apples" into 5 when a function wants an integer. OpenCart's own code does not use it; your modules should.

B.1.2 Arrays, which are really two things

```
<?php
// A list.
$sizes = ['S', 'M', 'L'];
echo $sizes[0];           // S
echo count($sizes);       // 3

// A map -- the same type, with keys. Almost every piece of data in
// OpenCart arrives as one of these.
$product = [
    'product_id' => 42,
    'name'       => 'Keyboard',
    'price'      => 2499.00,
];
echo $product['name'];    // Keyboard
```

```
// Walking one. Note the "as $key => $value" form.
foreach ($product as $field => $value) {
    echo "$field = $value\n";
}

// The ?? operator: use this value, or that one if it is absent.
// Without it, a missing key is a warning and a null.
$image = $product['image'] ?? 'placeholder.png';
```

B.1.3 Functions, classes and the arrow

```
<?php
function formatPrice(float $amount, string $symbol = 'Rs. '): string
{
    return $symbol . number_format($amount, 2);
}

class ShippingQuote
{
    private float $rate;

    public function __construct(float $rate)
    {
        $this->rate = $rate;           // $this-> reaches a property
    }

    public function forWeight(float $kg): float
    {
        return $this->rate * max(1.0, $kg);
    }
}

$q = new ShippingQuote(120.0);
echo formatPrice($q->forWeight(2.5)); // Rs. 300.00
```

💡 The three arrows, which are different

PHP uses three symbols that beginners merge into one.

- `$obj->method()` — call a method on an object, or read a property. An *object*.
- `$array['key']` — read a key from an array. An *array*. Square brackets, quoted key.
- `Class::CONSTANT` — reach something on the class itself rather than on an instance.

OpenCart hands you arrays far more often than objects, so `['key']` is the one you will type most. `$this->request->get['product_id']` is all three ideas in one expression: a property of a property, then an array key.

B.1.4 Where request data arrives, and why it is dangerous

```
<?php
// Submitted by a form with method="get" -> $_GET
// Submitted by a form with method="post" -> $_POST
$qty = $_POST['quantity'] ?? 1;

// EVERY value in those arrays is a STRING, and every one of them was
// typed by somebody who may not be a customer. Chapter 16 is about
// what happens when this is forgotten; Chapter 15 is about sessions,
// which is how the shop remembers who you are between requests.
$qty = (int)$qty; // now it is certainly an integer
```

```
if ($qty < 1 || $qty > 999) {
    $qty = 1;                // and certainly in range
}
```

B.2 SQL, as This Course Uses It

You met SQL in *Database Systems*. This section is a reminder of the five statements this course needs and the two ideas it leans on.

```
-- Read. The only one you will type often.
SELECT p.product_id, pd.name, p.price
FROM oc_product p
JOIN oc_product_description pd ON pd.product_id = p.product_id
WHERE pd.language_id = 1
AND p.status = 1
ORDER BY p.price DESC
LIMIT 20;

-- Count. Chapter 6 shows why a category page runs one of these as
-- well as the query above, every single time.
SELECT COUNT(*) FROM oc_product WHERE status = 1;

-- Change one row. The WHERE is not optional; without it you have
-- just repriced the entire catalogue.
UPDATE oc_product SET price = 2999.00 WHERE product_id = 42;

-- Remove. Same warning, more so.
DELETE FROM oc_product_to_category WHERE product_id = 42;

-- Make a query fast, or fail to -- see Chapter 6.
CREATE INDEX idx_sort_order ON oc_product (sort_order);
```

The two ideas everything else rests on

A **join** combines rows from two tables on a matching column. OpenCart splits a product across eighteen tables (Chapter 6), so almost every useful question needs at least one join.

An **index** is a sorted structure the database can search instead of reading every row. It makes reads faster and writes slower, and it is useless if the query hides the column inside a function — `ORDER BY LCASE(name)` cannot use an index on `name`, which is the central finding of Chapter 6.

EXPLAIN, which answers "why is this slow?"

Put `EXPLAIN` in front of any `SELECT` and the database tells you what it intends to do rather than doing it. Three words in the output carry nearly all the information:

- **type: ALL** — a full table scan. Every row read. At fifty thousand products this is the one to fix.
- **Using filesort** — the result had to be sorted after it was fetched, because no index gave it in the right order.
- **Using temporary** — a temporary table was built to hold intermediate results, usually because of `DISTINCT` or `GROUP BY`.

Chapter 6 runs `EXPLAIN` against the real category query and finds all three at once.

B.3 Twig, Which Is Just the Page

OpenCart's templates are Twig. A template is HTML with three kinds of hole in it, and that is very nearly the whole language.

```
{# A comment. It does not reach the browser. #}

{# 1. Print something. #}
<h1>{{ heading_title }}</h1>
<span class="price">{{ product.price }}</span>

{# 2. Do something -- a loop or a condition. #}
{% if products %}
<ul>
  {% for product in products %}
    <li>
      <a href="{{ product.href }}">{{ product.name }}</a>
      {% if product.special %}
        <s>{{ product.price }}</s> {{ product.special }}
      {% else %}
        {{ product.price }}
      {% endif %}
    </li>
  {% endfor %}
</ul>
{% else %}
<p>{{ text_empty }}</p>
{% endif %}

{# 3. Filter -- transform a value on the way out. #}
<p>{{ product.description|striptags|slice(0, 120) }}</p>
<p>{{ product.name|escape }}</p>
```

|escape and what it prevents

Twig escapes output by default in most configurations, and that default is the single thing standing between a product description written by somebody else and a cross-site scripting hole in your shop.

Twig also offers `|raw`, which turns the escaping off. There are legitimate uses, and every one of them is a decision to trust the source of that value completely. Chapter ?? looks for places where somebody has used it without meaning to.

B.4 YAML, in Ten Lines

The Compose file is YAML, and YAML has exactly two rules worth knowing.

```
# 1. Indentation is structure. Two spaces per level, NEVER tabs --
#    a tab is a syntax error and the message will not say so clearly.
services:
  db:
    image: mariadb:11
    environment:
      MARIADB_DATABASE: opencart

# 2. A leading dash is a list item.
ports:
  - "8090:80"
  - "3306:3306"
```

That is enough to read `code/docker-compose.yml` and to change the one thing Chapter 5 asks you to change, which is a port number.

B.5 Where To Look Next

For	Go to
PHP	php.net/manual — the official manual, and unusually good. Each function page has user-contributed notes; treat those as hints, not authority
SQL	The MariaDB knowledge base, and your <i>Database Systems</i> notes for the theory this appendix skips
Twig	twig.symfony.com/doc — the “for template designers” page is all you need
OpenCart	docs.opencart.com , and the source in your own container, which is more current than the documentation

💡 Read the source; it is right there

The shop’s entire source is inside your container, and you can read it:

```
docker compose exec shop \  
  cat /var/www/html/catalog/model/catalog/product.php
```

This is not a last resort. Chapter 6 finds the category page’s real query this way, and it is the only way to answer “what does this page actually do?” with certainty rather than with a plausible guess. The documentation describes a version; the container holds yours.

Answers to the Checkpoints

Three answers per chapter, in order. Attempt the checkpoint from memory before reading these, and argue with them where you disagree — several are judgement calls rather than facts, and they say so.

Chapter 1 — What E-Commerce Is, and What It Changed

1. **Electronic business, not electronic commerce.** Viewing results and downloading transcripts are business processes conducted over a network, but no value changes hands through the system — the transaction happens at a bank counter. The moment fee payment moves online it becomes e-commerce, and the university acquires every problem to the right of the dashed line in Figure 1.1: a payment that may have succeeded while the network dropped, a receipt that must exist exactly once, a refund path.
2. Many are acceptable. One good answer: **refusing to sell the last item when it is visibly damaged.** In software this requires a stock record that distinguishes “held” from “sellable”, somebody whose job is to update it, a rule for what the storefront does when the two disagree, and a decision about what the customer sees — out of stock, or a quiet substitution? The shopkeeper does all of that in half a second without recording that a decision was made, which is exactly why it is forgotten when the shop goes online.
3. **Two uncounted costs:** the acquisition cost of finding customers who currently arrive free (Chapter 2 puts this at Rs. 1,200 an order for a typical small shop, which can exceed the commission), and the labour of running the four functions themselves — payment integration, delivery arrangements, customer service, returns. The commission was buying those, not just the listing. **What they gain that is not money:** the customer relationship. They learn who bought, can contact them again, and cannot be delisted. For a business with repeat custom that is frequently worth more than the 15%.

Chapter 2 — Business Models and Revenue Models

1. **Business model:** a marketplace connecting restaurants with diners, with fulfilment performed in-house — three and a half of the four functions. **Revenue models: three at once** — *commission* from restaurants, a *service or delivery fee* from customers, and *advertising* in the form of promoted placement. **Most profitable per unit: the advertising.** It has almost no marginal cost, where commission is earned against riders and support that must be paid for, and the delivery fee rarely covers the rider. Promoted placement is close to pure margin, which is why these platforms push it so hard.
2. Viable when $LTV > CAC$, that is when $400 \times n > 1,500$, so when the average customer orders $n > 3.75$ — **at least four times** — before they stop. *What to measure:* the repeat rate and the time over which it happens. Four orders over six months is a business; four orders over six years is not, because the cash was spent today and returns over a period during which the venture has to survive.
3. B2C platforms store **one price per product** — a column on the product row. B2B pricing is a *relation* between customer and product, often with quantity breaks and expiry dates. That is not a setting you have failed to find; it is a different cardinality, and making it work means either a different platform or an extension that intercepts every price lookup in the system

(Chapter ?? shows how much work that is). The assumption that breaks is “a product has a price”.

Chapter 3 — Auctions, Portals and Communities

1. **Bid Rs. 15,000** — your true valuation. You win, and you pay the second-highest bid, **Rs. 13,500**, gaining Rs. 1,500. Bidding Rs. 18,000 does not improve this outcome at all: you still pay Rs. 13,500, because the price is set by the others. What it does is expose you to winning when somebody bids Rs. 16,500, at which point you pay Rs. 16,500 for a thing worth Rs. 15,000 to you. The overbid can only ever buy you an auction you did not want.
2. It **never closes**. The auction stays open indefinitely; the winning bidder is never notified, the seller is never paid, and the listing accumulates further bids whenever somebody wanders past. The seller sees an item that appears to be permanently “ending soon” and a buyer who has given up. Worse, the failure is *silent* and inversely proportional to interest: popular auctions close correctly and unpopular ones do not, so the bug is invisible in testing, where you load every page you create.
3. Any two of: **be a seller yourself** until real ones arrive (cost: you take on stock and inventory risk, and you compete with the sellers you are recruiting); **narrow the market** to one city or one category (cost: you cap your growth and may pick the wrong niche); **give sellers a tool worth having without buyers** (cost: you are now building and supporting a second product); **list supply gathered elsewhere** (cost: legally risky and reputationally expensive if discovered).

Chapter 4 — Planning an E-Commerce Framework

1. (a) **Functional**, and it *does not* discriminate — all four options reset passwords. (b) **Functional**, and it **discriminates sharply**: per-customer pricing eliminates most hosted platforms and the marketplace route outright, and is the single most common reason to choose self-hosted or bespoke. (c) **Non-functional**, and it discriminates *in the opposite direction* to what students expect — surviving a twentyfold spike is something a large hosted platform does effortlessly and your single virtual server does not.
2. **Hosting** (Rs. 324,000 in the worked example), **setup and theming labour** (Rs. 175,000 plus Rs. 100,000 of custom modules), and above all **maintenance labour** (Rs. 360,000). Their stated saving of Rs. 792,000 is therefore closer to Rs. 339,000 — still a saving, but less than half what they claimed, and it vanishes entirely if maintenance takes eight hours a month instead of four.
3. Many are acceptable. A clear one: **a home baker selling six kinds of cake to one city, taking orders through Instagram**. Self-hosting would mean paying for a server and maintaining it to serve perhaps thirty orders a week, with nobody to patch it — which is not a cheaper shop but an unpatched one. **Choose marketplace-only or a cheap hosted platform**: the business’s scarce resource is the baker’s time, the volume is low, nothing about the pricing is unusual, and every hour spent on a server is an hour not spent baking.

Chapter 5 — The Stack, and Your Shop in One Command

1. **Working**: the operating system and the web server — a 500 is generated and served by Apache, so the request reached it and a response came back. **Failed**: the application layer; PHP raised an error and died before producing a page. The database is *unknown* — it may be the cause (the application could not connect) or entirely healthy. **Look first** at the PHP error log, with `docker compose logs shop`; the message is never on the screen, because `display_errors` is off, and Chapter ?? is about why you want it that way.

2. **depends_on** orders container *starts*, and a container that has started is not a database that will accept a login — MariaDB opens its TCP port several seconds before it finishes initialising. The shop therefore starts, runs the installer, and the installer is refused by a database that is listening but not ready. **The failure is intermittent and worse on a fast machine**, because the faster the shop container starts, the more certainly it arrives early. The **service_healthy** condition waits for the database’s own healthcheck to pass, which tests a login rather than a port.
3. **Lost:** both named volumes — the entire database (products, categories, orders, customers, settings) and the shop’s installed files, including any module you had written in place. **Not lost:** anything in the repository — the Compose file, the Dockerfile, `code/`, and any module kept there rather than edited inside the container. **What would have made it minor:** a database dump taken on a schedule and kept outside the volume, plus the discipline of writing modules in `code/modules/` and mounting them, so that the only thing destroyed is state you can regenerate. Chapter ?? makes both of these routine.

Chapter 6 — Products, Categories and Their Tables

1. **Move to oc_product_description:** the product name and the meta description. Both are *words*, and words differ by language. **Stay in oc_product:** price, weight and stock quantity — a kilogram is a kilogram in Urdu, and a shop that let the price depend on the reader’s language would eventually charge two different prices for one product. The test is the first of the four questions: *does this fact depend on who is reading?*
2. **Buys:** a single-query answer to “every category beneath this one”, so a category page costs one query instead of a loop whose length depends on how deep the tree happens to be. **Costs:** the same fact is now stored twice, so the two can disagree, and every write that changes the shape of the tree must rewrite the derived rows. **The operation that makes it visible:** *moving a category*. Changing one **parent_id** invalidates the path rows of that category and of everything beneath it, so the admin panel must rewrite them all — which is why moving a category with many descendants is noticeably slower than renaming one.
3. Two of several good answers. (a) **The database was never the bottleneck:** if the database is 20% of the page, halving it improves the page by 10%, which can be smaller than the run-to-run variance. (b) **The expensive thing is the number of queries, not the cost of any one** — the measurement in this chapter found 146 queries for one page, and an index helps exactly one of them. **To tell them apart:** turn on the query log with `long_query_time=0`, request the page once, and read `COUNT(*)` and `SUM(query_time)` from `mysql.slow_log`. A large total with few queries is (a); a small total spread over a hundred queries is (b). That single measurement separates them, and it is the one in `m06_catalogue.py`.

Chapter 7 — Product Variations, Options and Uploads

1. **The photograph is an option** — specifically a **file** option. It is supplied by the customer, it is different every time, and the shop holds no stock of it. **The size is a variant**, because a small mug and a large mug are two different objects on two different shelves, each with its own cost, its own barcode and its own count. **What the shop holds in stock:** blank mugs in two sizes. Nothing else — which is exactly why the photograph cannot be a variant: there is no such thing as stock of “the mug with Ahmed’s photograph on it” until Ahmed orders one.
2. **Option value rows:** $4 + 3 + 2 = 9$, plus three rows in `oc_product_option`. **As variants:** $4 \times 3 \times 2 = 24$ combinations, so **25 oc_product rows** — twenty-four plus the master — and 25 description rows, 25 store rows and 25 category rows beside them. Nine against a hundred, for the same catalogue.

3. **Passes:** the filename sanitiser (nothing illegal in “logo.svg”), the length check, the extension allow-list (svg is on the stock list), and the MIME allow-list — it claimed `image/png`, which is allowed, and nothing verifies that claim. **Fails:** nothing. All five checks pass. **For it to harm a visitor**, the file must be *served to them from your origin and rendered*. Two things would have to be true: the attacker must learn the stored URL, which carries a 32-character random token and so is not guessable — but it *is* shown to whoever uploaded it and may be forwarded to an administrator who opens it — and the server must serve it with `Content-Type: image/svg+xml` rather than as a download, which Chapter 5’s stock install does. The realistic victim is therefore the administrator reviewing the order, and the realistic fix is to drop `svg` from the list and serve uploads with `Content-Disposition: attachment` (Chapter ??).

Chapter 8 — Themes, Layout and the Customer’s Experience

1. **Best case for compressing the photographs:** the images are $713 - 597 - 54 =$ about **61 KB**. Halving them saves 30 KB, which is 4% of the page — and that is the *best* case, because you cannot compress them to nothing. **Propose instead:** minify `bootstrap.css`, which is 265 KB of the 597 and has no minified copy in the tree. That single change saves roughly 240 KB, about **34%** of the page — eight times the whole image budget — and changes nothing a customer can see. Then remove the unused parts of Bootstrap and FontAwesome, and only then look at the images.
2. **Layout change** if a “trust badge” module already exists: **Design > Layouts > Product**, add the module to **Content Bottom**, set its sort order. No code, visible to the next person, survives upgrades. **Template override** if it must sit *immediately under the button*, inside the markup the theme generates: there is no position there, so you override `product/product.twig` in your child theme and add it. The general rule: a position that exists is a layout change; markup that must differ is a template change.
3. **Two reasons it is lower.** (a) **It follows only assets named in the HTML.** Fonts are requested by the CSS once the browser parses it, so none of the FontAwesome `.woff2` files — 248 KB in that directory — are counted. (b) **It does not execute JavaScript**, so anything a script loads afterwards (a tracking pixel, a lazy-loaded image, a carousel’s later slides) is invisible to it. **Which it could fix:** the first, by parsing the CSS for `url()` references and following them — a dozen lines. The second it cannot fix, because fixing it means running a browser, which is a different kind of tool. Knowing which limitation is laziness and which is fundamental is the useful judgement here.

Chapter 9 — The Shopping Basket, and Where State Lives

1. **What would change:** the two queries per request that the db engine spends reading and writing the session record. On this host that was worth about 20% of the page — 89 ms to 71 ms — when those two queries were removed altogether by the file engine, and Redis would remove them from the database in the same way. **What would not change: the basket.** It is rows in `oc_cart`, and the `Cart` rows column was 1 under both engines. **The number that settles it:** the session record is **18 bytes** and holds `{"currency": "USD"}`. Whatever you move it to, you are moving eighteen bytes. The right place to look for basket performance is the six cart queries an empty basket costs, not the store the session sits in.
2. **Where they came from:** a session row is written on every request, before the visitor has asked for anything, so every page view by every crawler, uptime check and measurement harness created one. The 645 rows here are the by-product of writing this handout. **Why they are not removed on departure:** nothing tells a server that a visitor has left — HTTP has no “goodbye”. Rows can therefore only expire on a clock, and `config_session_expire` is 86,400 seconds. **What to measure first:** the proportion of sessions that ever acquire a basket

or a login. If almost none do, the expiry is not the problem and robots are; shortening the expiry would then also shorten the life of the few baskets that matter, which is the opposite of what was wanted. Measure before tuning the number that is easiest to reach.

3. **What each removes:** `down` removes the containers — the running processes and their writable layers. The named volumes `dbdata` and `shopdata` are separate objects and are left alone, so the database files survive and the basket with them; `up -d` attaches the new containers to the same volumes and the shop resumes mid-visit. `down -v` additionally deletes those volumes, which is where every byte of durable state lives, so the entrypoint finds an empty database and reinstalls from scratch. **One other thing it destroys:** every product, category, order, customer and setting you have made since Chapter 5 — in this measurement the shop came back in 17 seconds looking perfectly healthy, with all of it gone. Uploaded images go with the `shopdata` volume at the same time.

The Sixteen-Week Schedule

This appendix is the **source of record** for the teaching schedule. The one-page `ECAD_Fall2026_Lecture_Schedule.pdf` distributed to students is generated from it, and the slide pipeline parses the table below. Change this first; regenerate the one-page version afterwards.

Twenty-three chapters are taught across sixteen weeks, so seven weeks carry two chapters. The pairings are not arbitrary. In every case the two chapters are either one topic the printed book splits for length — the catalogue and its variants, the two halves of the back office — or a pair in which the second is unusable without the first: a basket with nothing to check out, a gateway with no payment method chosen.

One pairing is deliberate in the other direction. **Chapter 16, Security, has a week to itself**, although it would pair naturally with Chapter 15. Its laboratory attacks the student’s own shop and then hardens it, and that does not fit in half a week alongside anything else.

D.1 The Schedule

Wk	Lecture topic	Chapters	Lab	Assessment
1	What e-commerce changed; business and revenue models	1, 2	Read four real shops; the arithmetic of one order	—
2	Auctions, portals and communities; planning a framework	3, 4	Three shapes compared; your requirements and your decision	—
3	The stack, and your shop in one command	5	Compose up; verify, break and reset the shop	—
4	Products, categories and the data model beneath them	6	50,000 products, and the index that rescues the category page	—
5	Product variations, options and user uploads	7	Option combinations, and what an upload field accepts	Quiz 1 (Ch 1–5)
6	Themes, layout and the user experience	8	Page weight before and after, measured in bytes and requests	Assignment 1 set
7	The shopping basket, and where state lives	9	Session, cookie and database baskets across a restart	—
8	The checkout and the order state machine	10	Midterm, then queries counted per checkout step	Midterm (Ch 1–9)

Wk	Lecture topic	Chapters	Lab	Assessment
9	Shipping, tax and zones; discounts and vouchers	11, 12	Geo zones and GST; a voucher applied in the wrong order	Assignment 1 due
10	Web payment systems; taking payment for orders	13, 14	A sandbox gateway, and what happens when it times out	Project proposal
11	User account management	15	Password cost against logins per second	—
12	Securing the shop	16	The OWASP checks against your own container, before and after hardening	Quiz 2 (Ch 10–15)
13	Administration: the dashboard, orders, customers, refunds	17, 18	A refund through the order state machine	Assignment 2 set
14	Extending the platform: your first module	19	A shipping method and a dashboard widget, written from nothing	—
15	Deployment, maintenance and performance; SEO	20, 21	Load test with cache on and off; crawlability measured	Assignment 2 due
16	Social, legal and ethical issues; project defence	22, 23	Project presentations	Project due; presentations

D.2 How the Assessment Adds Up

HEC publishes the assessment instruments for this course — sessional examination, home assignments, quizzes, project, presentations, final examination — without weights. The weights below are the department's, and are stated here so that a student can see the whole shape of the semester on one page.

Instrument	Weight	What it examines
Two quizzes	10%	Weeks 5 and 12. Short, closed-book, on the chapters named in the schedule
Two assignments	15%	Set in weeks 6 and 13. Both are done on your own shop and submitted as a change to it plus a short write-up
Midterm examination	20%	Week 8, Chapters 1–9
Semester project	25%	The shop itself, built across the whole semester (Chapter ??), submitted in week 16
Presentation and defence	5%	Week 16. Ten minutes, plus questions you are expected to be able to answer about your own decisions
Final examination	25%	All chapters, with emphasis after the midterm

⚠ The project is not a week-16 activity

Twenty-five per cent of the marks are for a shop that is built in the laboratory of every chapter from Chapter 5 onward. A student who treats the laboratories as optional homework and intends to build the project in the last fortnight will be building, in two weeks and without help, something the rest of the class built in fourteen weeks with a lecturer in the room.

Chapter ?? states what is required and how it is marked. Read it in Week 3, not in Week 15.